ACADEMIC EXCELLENCE SERIES • RAIT 2026

OBJECT ORIENTED PROGRAMMING

Complete End-Semester Examination Master Preparation Guide & Code Compendium (Java)

Institution: D. Y. Patil University • Ramrao Adik Institute of Technology (RAIT), Navi Mumbai

Departments: Department of Computer Engineering & Department of Artificial Intelligence & Data Science

Course Codes: 2413CEC3T1 / ADC303 / 231CEUCC32 (Semester III)

Source Foundation: Official Course Syllabus, 400+ Slide Lecture Series, Lab Manual & 2023–2026 PYQs

100% Syllabus & PYQ Coverage

All 6 Units covered with theoretical proofs, diagrams, and exact university question patterns.

High-Frequency Question Matrix

Prioritized by historical recurrence (7x, 6x, 5x, 4x) across multiple exam cycles.

20+ Ready-to-Run Java Codes

Fully compiled, standard-compliant Java programs with user input, dry runs, and outputs.

Examiner Marking Checklists

Scoring guidelines for securing full 5/5 and 10/10 marks in theoretical and design questions.

COURSE BLUEPRINT & SYLLABUS ARCHITECTURE

1. Course Schema & Evaluation Framework

The Object Oriented Programming course is a fundamental pillar of the B.Tech Computer Engineering and AI&DS curricula at D. Y. Patil University (RAIT). It bridges fundamental computational thinking with robust, enterprise-grade software development principles using the Java programming language.

COURSE CODE	COURSE TITLE	TEACHING SCHEME (HOURS)	CREDITS ASSIGNED	INTERNAL ASSESSMENT	END SEMESTER EXAM	TOTAL MARKS
2413CEC3T1	Object Oriented Programming (Theory)	03 Hours / Week (39 Contact Hours)	03 Credits	50 Marks (Best 2 CA Tests + Quizzes/ Assignments)	**60 / 100 Marks** (2 or 3 Hours)	100 / 150 Marks
2413CEC3L1	Object Oriented Programming Lab	02 Hours / Week (26 Contact Hours)	01 Credit	30 Marks (Continuous Evaluation + Quiz + Attendance)	**30 Marks** (Practical & Oral Exam)	60 Marks

2. Course Outcomes (CO) Mapping

CO CODE	COURSE OUTCOME DESCRIPTION	MAPPED MODULES	BLOOM'S TAXONOMY LEVEL
CO1	Understand the basics of OOP and apply fundamental Java programming constructs.	Module 1	BT1 (Remember), BT2 (Understand), BT4 (Analyze)
CO2	Understand and illustrate the features of classes, objects, constructors, and parameter passing.	Module 2	BT2 (Understand), BT3 (Apply), BT6 (Create)
CO3	Elaborate and apply the concepts of arrays, matrices, and String / StringBuffer classes.	Module 3	BT2 (Understand), BT3 (Apply), BT6 (Create)
CO4	Develop programs implementing inheritance hierarchies, abstract classes, interfaces, and packages.	Module 4	BT3 (Apply), BT4 (Analyze), BT6 (Create)
CO5	Implement robust software solutions utilizing exception handling and multithreading concurrency.	Module 5	BT2 (Understand), BT3 (Apply), BT6 (Create)
CO6	Design and develop graphical user interfaces (GUI) utilizing Java Swing and AWT layout managers.	Module 6	BT2 (Understand), BT3 (Apply), BT6 (Create)

3. End-Semester Examination Structure & Marks Distribution

Across the 2023–2026 examination papers (60-mark and 100-mark formats), the paper follows a structured question selection model designed to evaluate both theoretical clarity and hands-on coding proficiency:

- **Question 1 (20 Marks):** Compulsory / Broad Selection (Solve any 4 out of 6 questions, 5 marks each). Emphasizes core conceptual definitions, difference tables, and short programs (e.g., OOP pillars, access specifiers, String methods, super keyword, thread lifecycle, AWT vs Swing).
- **Question 2 (20 Marks):** Core Analytical & Short Coding (Solve any 4 out of 6 questions, 5 marks each). Evaluates class implementation, recursion, arrays/matrices, method overloading/overriding, multiple catch blocks, and GUI layouts.
- **Question 3 (20 Marks):** Comprehensive Application Design (Solve any 2 out of 3 questions, 10 marks each). High-weightage design and implementation questions:
    - University Sports Grace Marks system using Interface & Base class (CO4, BT6).
    - Custom Exception for odd number validation while calculating square (CO5, BT6).
    - Argument passing mechanism (primitives vs objects) with complete dry-run trace (CO2, BT4).
    - Volume of sphere using Multilevel Inheritance (CO4, BT4).
    - Multithreading concurrency (Odd and Even number generation) (CO5, BT3).
- **Question 4 (In 100-Mark Scheme - 20 Marks):** Abstract class hierarchies (e.g., Solid class with Cylinder and Sphere) and detailed thread lifecycle illustrations.

4. Master Question Frequency Analysis (Historical PYQ Recurrence)

The following table synthesizes the exact question recurrence extracted from official DY Patil RAIT examination papers and departmental question banks. Priority 1 items must be mastered first.

RANK	QUESTION TOPIC / DESCRIPTION	MAPPED MODULE	HISTORICAL RECURRENCE	TYPICAL MARKS
1	OOP Paradigm, Core Features & The 4 Pillars (Encapsulation, Abstraction, Inheritance, Polymorphism)	Module 1	**7 Times** (PYQ + QB)	5 / 10 Marks
2	Divide by Zero Exception handling using try-catch blocks with user input	Module 5	**6 Times** (PYQ + QB)	5 Marks
3	Difference between AWT and Swing packages in Java	Module 6	**6 Times** (PYQ + QB)	5 Marks
4	Access Specifiers in Java (public, private, protected, default) with scope matrix and examples	Module 2	**5 Times** (PYQ + QB)	5 Marks
5	Importance and Types of Inheritance (Single, Multilevel, Hierarchical, Multiple)	Module 4	**5 Times** (PYQ + QB)	5 / 8 Marks
6	Layout Managers in Java GUI / Swing (FlowLayout, BorderLayout, GridLayout, etc.)	Module 6	**5 Times** (PYQ + QB)	5 Marks
7	University Sports Grace Marks Application using Interface & Base Class	Module 4	**4 Times** (PYQ + QB)	10 Marks
8	Volume of Sphere using Multilevel Inheritance (Radius → Circle → Sphere)	Module 4	**4 Times** (PYQ + QB)	10 Marks
9	Constructor Overloading concept and Area calculation program	Module 2	**4 Times** (PYQ + QB)	5 / 10 Marks
10	Argument Passing Mechanism (Pass-by-value vs Reference) & Object as Argument / Return Object	Module 2	**4 Times** (PYQ + QB)	10 Marks
11	Thread Lifecycle in Java with complete state transition diagram	Module 5	**4 Times** (PYQ + QB)	5 / 10 Marks
12	Thread Creation: Extending Thread class vs. Implementing Runnable interface	Module 5	**4 Times** (PYQ + QB)	5 / 10 Marks
13	Multithreading Program (Concurrent Odd and Even numbers generation)	Module 5	**4 Times** (PYQ + QB)	5 / 10 Marks
14	String Class methods vs StringBuffer Class methods and differences	Module 3	**4 Times** (PYQ + QB)	5 Marks
15	Structured Programming (C) vs Object-Oriented Programming (Java)	Module 1	**3 Times** (PYQ + QB)	5 Marks
16	Class Rectangle program: calculate area and perimeter using constructor	Module 2	**3 Times** (PYQ + QB)	5 Marks
17	Method Overloading vs Method Overriding (rules, signatures, binding)	Module 4	**3 Times** (PYQ + QB)	5 Marks
18		Module 3	**3 Times** (PYQ + QB)	5 Marks

RANK	QUESTION TOPIC / DESCRIPTION	MAPPED MODULE	HISTORICAL RECURRENCE	TYPICAL MARKS
	String Palindrome verification using backwards/ forwards traversal (StringBuffer)			
19	Sum and Average of Array elements with user input	Module 3	**3 Times** (PYQ)	5 Marks
20	m × n Matrix acceptance and natural matrix form display / Transpose	Module 3	**3 Times** (PYQ + QB)	5 Marks
21	Fibonacci Series and Factorial of a number using Recursion	Module 2 / 3	**3 Times** (PYQ + QB)	5 Marks
22	Custom User-Defined Exception: OddNumberException for squaring even numbers	Module 5	**2 Times** (PYQ + QB)	10 Marks
23	JVM Architecture detailed working with diagram (ClassLoader, Memory, Execution Engine)	Module 1	**2 Times** (PYQ + QB)	5 Marks
24	The `super` and `final` keywords (all 3 uses of each with code)	Module 4	**2 Times** (PYQ + QB)	5 Marks
25	Java Swing Class Hierarchy diagram and container concepts	Module 6	**2 Times** (PYQ + QB)	5 Marks

5. Examiner Scoring Strategy: How to Secure 100% Marks

GOLDEN RULES FOR EXAM WRITING

University examiners follow a rigorous rubric. To secure maximum marks, every answer should adhere to the following 5-point structure:

1. **Formal Definition & Synopsis (1 Mark):** Begin with a crisp, 2-line technical definition. Never start directly with code.
2. **Syntax / Method Signature (1 Mark):** Clearly write the exact Java language syntax and keyword rules.
3. **Architectural Diagram / Memory Visualization (1–2 Marks):** Draw neat, labeled block diagrams (e.g., JVM subsystems, memory stack vs heap, thread state transitions, class hierarchy).
4. **Clean, Commented Java Code (2–4 Marks):** Write syntactically valid code including package imports (`import java.util.Scanner;`), class declaration, constructor, and `main()` driver method. Avoid pseudocode.
5. **Dry-Run Table & Sample Output (1 Mark):** Provide an execution trace table and exact terminal input/output format.

MODULE 1: INTRODUCTION TO OOP & JAVA FUNDAMENTALS

FREQUENCY: 7X (PYQ + QB) | PYQ 2023, 2024, 2025, 2026 | MARKS: 5 / 10 | CO1 (BT4)

Q1.1: Explain the Object-Oriented Programming Paradigm, its Core Features, and the Four Pillars of OOP in detail.

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects", which contain data in the form of fields (often known as attributes or properties) and code in the form of procedures (often known as methods). Unlike procedural programming which focuses on algorithms and actions, OOP organizes software design around data and modular entities that model real-world interactions.

The Four Pillars of OOP

```
+-------------------------------------------------------------------------------+ | THE FOUR PILLARS OF OOP | +-------------------------+-------------------------
+-------------------------------+ | 1. ENCAPSULATION | 2. ABSTRACTION | 3. INHERITANCE | | - Data Hiding | - Essential features | - Code Reusability | | - Bundling data + code | - Hiding implementation | - IS-A relationship | | - Getters / Setters | - Abstract class/IFace | - Extensibility | +-------------------------+-------------------------
+-------------------------------+ | 4. POLYMORPHISM | | - Compile-time (Method Overloading) & Run-time (Method Overriding) | | - "Many forms" - single interface, multiple underlying implementations |
+-------------------------------------------------------------------------------+
```

Figure 1.1: The Four Foundational Pillars of Object-Oriented Programming

1. **Encapsulation (Data Hiding):** Encapsulation is the mechanism of wrapping data (variables) and code acting on the data (methods) together as a single unit (class). In Java, encapsulation is achieved by declaring instance variables as `private` and providing public getter and setter methods to access and modify them.
   Benefits: Data security, maintainability, and controlled access through validation logic.
2. **Data Abstraction:** Abstraction is the quality of dealing with ideas rather than events. It represents only the essential features of an entity without including the background implementation details or explanations. In Java, abstraction is achieved using **Abstract Classes** (0 to 100% abstraction) and **Interfaces** (100% abstraction).
   Real-world analogy: When driving a car, the driver knows that pressing the accelerator increases speed, without needing to know the internal combustion or electronic throttle mechanics.
3. **Inheritance:** Inheritance is the mechanism by which one class acquires the properties (fields) and behaviors (methods) of another class. It establishes an **IS-A** relationship (e.g., a `Car` IS-A `Vehicle`). The class whose features are inherited is called the *Superclass* (Parent/Base), and the inheriting class is called the *Subclass* (Child/Derived).
   Benefits: Maximum code reusability, modular hierarchy, and polymorphism enablement.

4. **Polymorphism:** Derived from the Greek words *poly* (many) and *morph* (forms), polymorphism is the ability of an entity (such as a method, operator, or object) to take on different forms depending on the context.
    - **Compile-time Polymorphism (Static Binding):** Achieved via *Method Overloading* where multiple methods share the same name with different parameter signatures.
    - **Run-time Polymorphism (Dynamic Method Dispatch):** Achieved via *Method Overriding* where a subclass provides a specific implementation of a method already defined in its superclass, resolved at runtime via the object reference.

Core Features of OOP

- **Class & Object:** A class is a blueprint/template; an object is a runtime physical instance of that blueprint.
- **Data Hiding:** Preventing direct outside access to internal data members through access modifiers.
- **Dynamic Binding:** Linking a procedure call to the code to be executed only at runtime.
- **Message Passing:** Objects communicate with each other by sending and receiving messages (method invocations with parameters).

Examiner Marking Checklist (5/5 or 10/10 Score)

- Definition of OOP paradigm (1 Mark)
- Clear breakdown of all 4 pillars with accurate technical keywords (2 Marks)
- Diagram illustrating the 4 pillars (1 Mark)
- Distinction between compile-time and runtime polymorphism (1 Mark)

FREQUENCY: 3X (PYQ + QB) | PYQ 2023, 2025 | MARKS: 5 | CO1 (BT4)

Q1.2: Compare and contrast Structured/Procedural Programming (C) and Object-Oriented Programming (Java).

FEATURE / PARAMETER	STRUCTURED / PROCEDURAL PROGRAMMING (E.G., C)	OBJECT-ORIENTED PROGRAMMING (E.G., JAVA)
Core Focus	Focuses on functions, procedures, and sequence of actions (algorithms).	Focuses on data, state, and objects that encapsulate both data and behaviors.
Design Approach	**Top-Down Approach**: Solves problems by breaking down main tasks into smaller sub-functions.	**Bottom-Up Approach**: First models real-world entities as objects and builds applications upward.
Data Security	Low security. Data flows freely between functions via global variables.	High security. Data is hidden inside objects using access modifiers (`private`).
Code Reusability	Limited to function calls. Modifying a function may break dependent modules.	Extremely high through **Inheritance** and polymorphism without altering base code.
Access Specifiers	Does not support access modifiers (everything is accessible in scope).	Supports strict access specifiers: `private` , `default` , `protected` , `public` .
Overloading & Polymorphism	Not supported. Every function must have a completely unique name.	Fully supported via Compile-Time (Overloading) and Run-Time (Overriding) Polymorphism.
Memory Management	Manual allocation and deallocation (`malloc()` , `free()`). Prone to memory leaks.	Automatic Garbage Collection (GC) managed by the Java Virtual Machine.
Real-World Modeling	Difficult to model real-world complex entities.	Directly models real-world entities through classes, attributes, and relationships.

FREQUENCY: 2X (PYQ + QB) | PYQ 2024, 2026 | MARKS: 5 | CO1 (BT2)

Q1.3: Compare Java and C++ Programming Languages in detail.

FEATURE	C++	JAVA
Platform Dependency	Platform Dependent. Compiled directly into native machine code (object code).	**Platform Independent**. Compiled into intermediate Bytecode executed by JVM.
Pointers	Supports explicit pointers and direct memory address arithmetic.	Does **not** support explicit pointers or pointer arithmetic (enhances security).
Multiple Inheritance	Supports multiple inheritance through classes directly (leads to Diamond problem).	Does not support multiple inheritance with classes; achieved safely via **Interfaces**.
Memory Management	Manual memory management using `new` and `delete` operators.	Automatic Garbage Collection handles deallocation of unreachable objects.
Operator Overloading	Fully supported (operators like `+`, `-` can be redefined).	Not supported for user-defined types (only `+` is internally overloaded for String concat).
Execution Model	Compiled language only (Source → Machine binary).	Both **Compiled and Interpreted** (Source → Bytecode → JIT/Interpreter → Machine code).
Preprocessor Directives	Uses `#include`, `#define` preprocessor macros.	Uses `import` statements; no preprocessor or macro substitution.
Thread Support	External libraries or C++11 `<thread>`.	Built-in multithreading support directly in core language (`java.lang.Thread`, `Runnable`).

FREQUENCY: 2X (PYQ + QB) | PYQ 2024 | MARKS: 5 / 8 | CO1 (BT2)

Q1.4: Explain the detailed working of JVM (Java Virtual Machine) architecture with a neat diagram.

The **Java Virtual Machine (JVM)** is an abstract virtual computer that provides the runtime execution environment for compiled Java bytecode (`.class` files). It is responsible for loading, verifying, executing bytecode, and managing system memory.

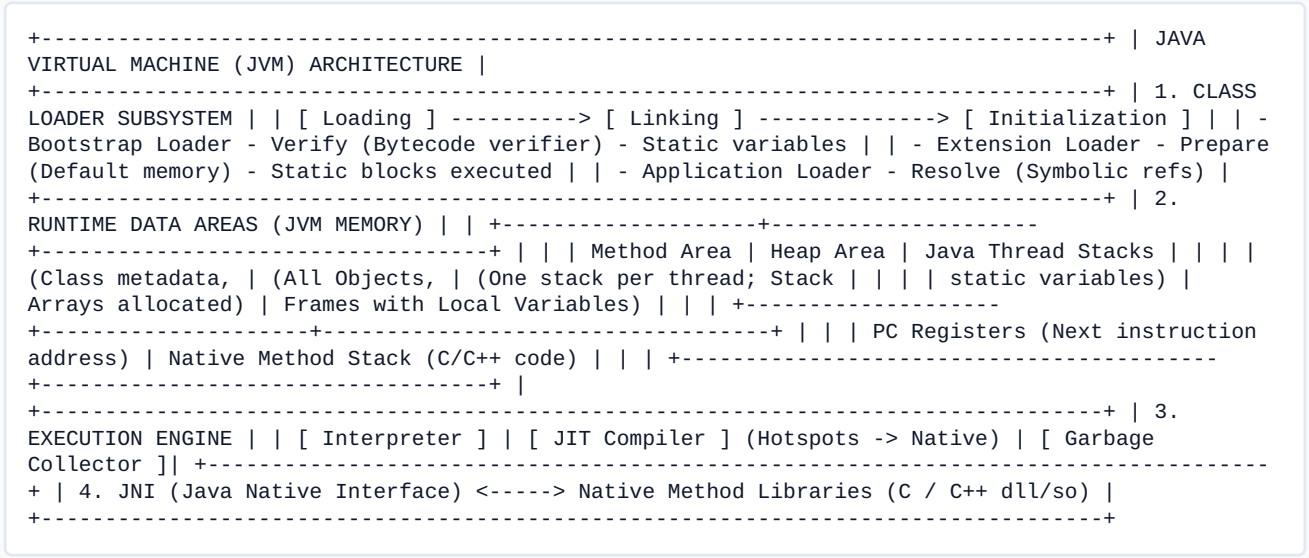


Figure 1.2: Detailed Architectural Subsystems of the Java Virtual Machine (JVM)

Key Architectural Subsystems of JVM

1. **Class Loader Subsystem:** Responsible for locating and importing binary data for types (classes and interfaces).
    - **Loading:** Bootstrap ClassLoader (loads `rt.jar`), Extension ClassLoader (loads from `jre/lib/ext`), and Application/System ClassLoader (loads from application classpath).
    - **Linking:**
        - *Verification:* Checks bytecode against Java language constraints (prevents stack overflow, invalid type casts).
        - *Preparation:* Allocates memory for static fields and initializes them to default values.
        - *Resolution:* Replaces symbolic references in the constant pool with direct memory references.
    - **Initialization:** Executes static initializers and static blocks in hierarchical order.
2. **Runtime Data Areas (JVM Memory):**
    - **Method Area (Metaspace):** Shared across all threads; stores per-class structures such as runtime constant pool, field/method data, and static variables.
    - **Heap Area:** Shared across all threads; runtime data area from which memory for all class instances and arrays is allocated. Garbage Collector operates here.
    - **Java Thread Stacks:** Private to each thread; created concurrently with the thread. Stores frames containing local variables, partial results, and method invocation return addresses.
    - **PC (Program Counter) Registers:** Private to each thread; contains the address of the JVM instruction currently being executed.
    - **Native Method Stacks:** Supports native methods written in languages such as C or C++.
3. **Execution Engine:**
    - **Interpreter:** Reads bytecode stream line-by-line and executes native commands. Fast to start, but slower in execution of repeated code.
    - **Just-In-Time (JIT) Compiler:** Identifies frequently executed code blocks ("hotspots") and compiles the entire bytecode block directly into native machine code. Subsequent calls execute natively at hardware speed.
    - **Garbage Collector (GC):** Automatically tracks object references and reclaims memory allocated to objects that are no longer reachable from any live thread.

4. **Java Native Interface (JNI):** A bridge framework that allows Java code to interact with native libraries written in C, C++, or Assembly.

FREQUENCY: 2X (PYQ) PYQ 2023, 2026 MARKS: 4 / 5 CO1 (BT2)

Q1.5: Explain how Java achieves Platform Independence ("Write Once, Run Anywhere") and Write a Short Note on JDK, JRE, and JVM.

Platform Independence in Java

In traditional languages like C/C++, source code is compiled directly into platform-specific machine code (e.g., x86 binary). Hence, an executable compiled for Windows will fail to run on Linux or macOS.

Java solves this through a two-step execution pipeline:

1. The **Java Compiler (`javac`)** compiles human-readable source code (`.java`) not into machine code, but into an optimized, highly compressed intermediate instruction set called **Bytecode (`.class` files)**.
2. Bytecode is completely neutral and independent of underlying operating systems or hardware architectures.
3. The **Java Virtual Machine (JVM)** is platform-specific. Sun/Oracle provides distinct JVM implementations for Windows, Linux, and macOS. Any machine that has a compatible JVM installed can execute the exact same bytecode file.

Thus, Java embodies the famous philosophy: *"Write Once, Run Anywhere" (WORA)*.

JDK vs. JRE vs. JVM Architecture

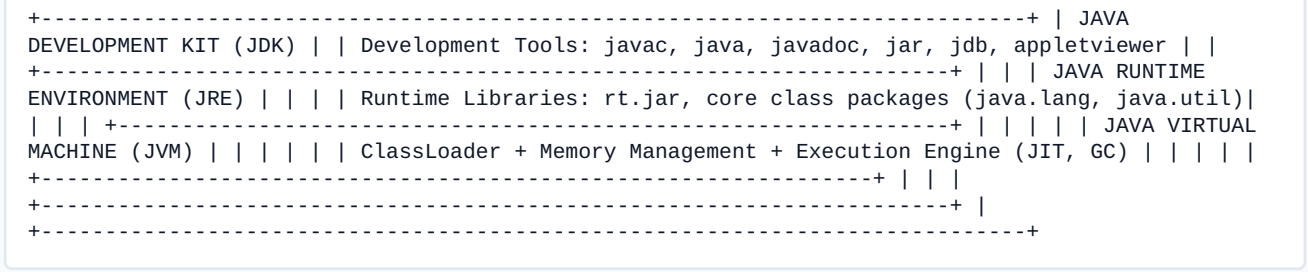


Figure 1.3: Nested Relationship of JDK, JRE, and JVM

- **JVM (Java Virtual Machine):** The core execution engine. It is an abstract runtime environment that converts bytecode into native machine instructions. It has no development tools.
- **JRE (Java Runtime Environment):** The minimum software package required to *run* an already compiled Java program.

  ```
  JRE = JVM + Core Runtime Class Libraries
  ```

- **JDK (Java Development Kit):** The complete software development package required to *develop, compile, debug, and run* Java applications.

  ```
  JDK = JRE + Development Tools (javac, java, jdb, jar)
  ```

FREQUENCY: 1X (PYQ) | PYQ 2024 CAC303 Q1(D) | MARKS: 5 | CO1 (BT6)

Q1.6: Write a Java program to display your Name and Age with user input, and explain the anatomy of `public static void main(String[] args)` and `System.out.println()`.

The standard entry point of any standalone Java application is the `main()` method. Its signature is rigorously enforced by the Java Virtual Machine launcher:

- **`public`:** Access specifier that grants unrestricted visibility, permitting the external JVM runtime to locate and invoke the method from outside the enclosing package.
- **`static`:** Allows the method to be invoked without first instantiating an object of the class (the class is loaded into memory, and `main()` is invoked directly on the class).
- **`void`:** Return type specifying that the method does not return a value upon termination.
- **`main`:** The fixed identifier sought by the JVM as the initial execution entry point.
- **`String[] args`:** Array of `String` objects representing command-line arguments passed into the program upon launch.
- **`System.out.println()`:** `System` is a final class in `java.lang`; `out` is a public static instance of `PrintStream`; `println()` prints the passed argument to standard console output followed by a platform-specific newline.

```
// Program: Display Name and Age with Interactive Console Input
import java.util.Scanner;

public class DisplayNameAge {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your Full Name: ");
        String name = sc.nextLine();

        System.out.print("Enter your Age: ");
        int age = sc.nextInt();

        System.out.println("\n--- Personal Profile Summary ---");
        System.out.println("Student Name : " + name);
        System.out.println("Student Age  : " + age + " years");

        sc.close();
    }
}
```

```
Enter your Full Name: Pranav Patil Enter your Age: 20 --- Personal Profile Summary --- Student Name :
Pranav Patil Student Age : 20 years
```

FREQUENCY: 2X (PYQ) PYQ 2024, 2026 MARKS: 5 CO1 (BT2)

Q1.7: Describe the concept of Type Casting and Type Conversion in Java with syntax and working code examples.

Assigning a value of one primitive data type to another is called **Type Casting** or **Type Conversion**. In Java, there are two distinct categories:

1. Widening / Implicit Type Conversion (Automatic)

- Occurs automatically when converting a smaller datatype into a larger datatype.
- The two types must be compatible, and the destination type must have a larger byte size than the source type.
- There is **zero loss of precision or data**.
- *Conversion Hierarchy:* `byte (1B) → short (2B) → int (4B) → long (8B) → float (4B) → double (8B)`

2. Narrowing / Explicit Type Casting (Manual)

- Required when converting a larger datatype into a smaller datatype.
- Must be done explicitly by the programmer using the cast operator `(target_type)`.
- Can result in **data truncation, overflow, or loss of decimal precision**.
- *Hierarchy:* `double (8B) → float (4B) → long (8B) → int (4B) → short (2B) → byte (1B)`

```
// Program: Demonstration of Type Conversion and Type Casting in Java
public class TypeCastingDemo {
    public static void main(String[] args) {
        // --- 1. Widening Conversion (Implicit) ---
        int intVal = 100;
        long longVal = intVal;       // Automatic: int (4 bytes) to long (8 bytes)
        double doubleVal = longVal;  // Automatic: long to double

        System.out.println("--- Widening Conversion (Implicit) ---");
        System.out.println("Integer value : " + intVal);
        System.out.println("Long value    : " + longVal);
        System.out.println("Double value  : " + doubleVal);

        // --- 2. Narrowing Casting (Explicit) ---
        double pi = 3.1415926535;
        int truncatedPi = (int) pi;   // Manual cast: fractional part truncated

        int largeInt = 130;
        byte byteVal = (byte) largeInt; // Range of byte is -128 to 127; causes overflow wrap

        System.out.println("\n--- Narrowing Casting (Explicit) ---");
        System.out.println("Original double: " + pi);
        System.out.println("Cast to int    : " + truncatedPi + " (fractional precision
lost)");
        System.out.println("Original int   : " + largeInt);
        System.out.println("Cast to byte   : " + byteVal + " (wrapped: 130 - 256 = -126)");
    }
}
```

```
--- Widening Conversion (Implicit) --- Integer value : 100 Long value : 100 Double value : 100.0 ---
Narrowing Casting (Explicit) --- Original double: 3.1415926535 Cast to int : 3 (fractional precision
lost) Original int : 130 Cast to byte : -126 (wrapped: 130 - 256 = -126)
```

FREQUENCY: CORE SYLLABUS | LECTURE NOTES & LAB EXP 1 | MARKS: 5 | CO1 (BT3)

Q1.8: Compare `Scanner` class and `BufferedReader` class for User Input in Java with code examples.

PARAMETER	SCANNER CLASS (`JAVA.UTIL.SCANNER`)	BUFFEREDREADER CLASS (`JAVA.IO.BUFFEREDREADER`)
Introduction	Introduced in Java 1.5 for easy tokenized parsing of primitive types.	Classic I/O class present since Java 1.1 for high-speed buffered character stream reading.
Parsing Capability	Parses data into primitives directly (`nextInt()`, `nextDouble()`, `next()`).	Reads only text strings via `readLine()`; requires manual parsing (e.g., `Integer.parseInt()`).
Buffer Size & Speed	Smaller buffer (1 KB). Slower due to regex pattern matching during parsing.	Large default buffer (8 KB). Significantly faster for massive data input streams.
Thread Safety	Not synchronized (not thread-safe).	Synchronized methods (thread-safe).
Exception Handling	Throws unchecked `InputMismatchException`; does not mandate try-catch for basic console reading.	Mandates handling checked `IOException` (either via `try-catch` or `throws IOException`).

```
// Program: Demonstration of User Input via Scanner and BufferedReader
import java.util.Scanner;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class InputComparisonDemo {
    public static void main(String[] args) throws IOException {
        // --- 1. Using Scanner ---
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter an integer using Scanner: ");
        int num1 = sc.nextInt();
        System.out.println("Scanner captured: " + num1);

        // --- 2. Using BufferedReader ---
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        System.out.print("Enter an integer using BufferedReader: ");
        String str = br.readLine();
        int num2 = Integer.parseInt(str); // Manual parsing required
        System.out.println("BufferedReader captured: " + num2);
    }
}
```

MODULE 2: CLASSES, OBJECTS, METHODS & CONSTRUCTORS

FREQUENCY: 1X (PYQ) | PYQ 2023 | MARKS: 5 | CO2 (BT2)

Q2.1: Explain the concept of 'Class' and 'Object' in Java with syntax, memory allocation, and an illustrative example.

Class: A class is a user-defined blueprint, prototype, or template from which individual objects are created. It represents the set of properties (state) and methods (behavior) that are common to all objects of one type. A class is a logical construct that consumes no heap memory until instantiated.

Object: An object is an identifiable, concrete instance of a class. It has state (stored in fields), behavior (exposed via methods), and a distinct identity (unique memory address). Objects are dynamically allocated on the **Java Heap** at runtime via the `new` keyword.

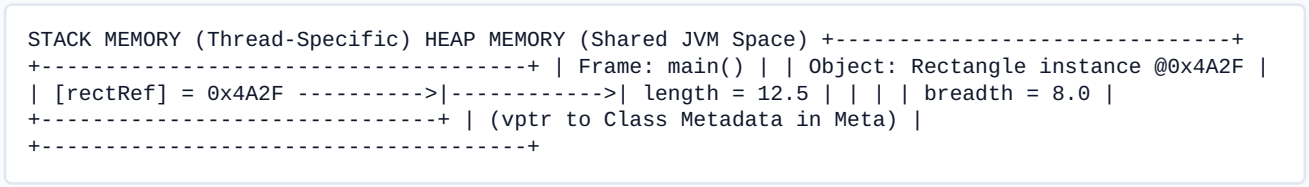


Figure 2.1: Stack Reference pointing to Dynamic Heap Object Memory

```
// Definition of Class
class Student {
    // State / Instance Variables
    String name;
    int rollNo;

    // Behavior / Method
    void setDetails(String n, int r) {
        name = n;
        rollNo = r;
    }

    void display() {
        System.out.println("Student Name: " + name + ", Roll No: " + rollNo);
    }
}

// Driver Class demonstrating Object Instantiation
public class ClassObjectDemo {
    public static void main(String[] args) {
        // Declaration and dynamic allocation
        Student s1 = new Student(); // 's1' is reference on stack, object is in heap
        s1.setDetails("Aditya Sharma", 101);
        s1.display();
    }
}
```

FREQUENCY: 5X (PYQ + QB) PYQ 2023, 2024, 2025, 2026 MARKS: 5 CO2 (BT2)

Q2.2: Explain the different Access Specifiers in Java (public, private, protected, default) with scope visibility matrix and code examples.

Access specifiers (modifiers) in Java determine the accessibility or scope of classes, constructors, methods, and variables. Java provides four distinct levels of access control:

1. `private`**:** Most restrictive. Accessible *only within the same class*. Neither subclasses nor other classes in the same package can access private members. Used to enforce data encapsulation.
2. `default` **(Package-Private / No modifier):** When no access specifier is specified, Java assigns default access. Accessible *only within classes of the same package*. Invisible outside the package, even to subclasses.
3. `protected`**:** Accessible *within the same package* AND across different packages *only through inheritance (subclasses)*. Cannot be accessed by unrelated classes in external packages.
4. `public`**:** Least restrictive. Accessible from *any other class* in any package across the entire application classpath.

Access Visibility Matrix

ACCESS MODIFIER	SAME CLASS	SAME PACKAGE SUBCLASS	SAME PACKAGE NON-SUBCLASS	DIFFERENT PACKAGE SUBCLASS	DIFFERENT PACKAGE NON-SUBCLASS
`private`	YES	NO	NO	NO	NO
`default`	YES	YES	YES	NO	NO
`protected`	YES	YES	YES	YES (via inheritance)	NO
`public`	YES	YES	YES	YES	YES

```
// Demonstration of Access Specifiers
public class AccessSpecifierDemo {
    private int privateVar = 10;       // Only within this class
    int defaultVar = 20;               // Package-private
    protected int protectedVar = 30;   // Package + Subclasses
    public int publicVar = 40;         // Everywhere

    public void showAccess() {
        System.out.println("Private   : " + privateVar);
        System.out.println("Default   : " + defaultVar);
        System.out.println("Protected : " + protectedVar);
        System.out.println("Public    : " + publicVar);
    }
}
```

FREQUENCY: 2X (PYQ + QB) | PYQ 2025, 2026 | MARKS: 5 | CO2 (BT2)

Q2.3: Elaborate the different types of variables used in Java (Local, Instance, Static/Class) and explain Static data members and methods.

PARAMETER	LOCAL VARIABLE	INSTANCE VARIABLE	STATIC / CLASS VARIABLE
Declaration Location	Inside a method, constructor, or block.	Inside a class, but outside any method or constructor.	Inside a class with the `static` keyword, outside methods.
Scope	Limited to the method/ block where declared.	Accessible by all non-static methods of the class.	Accessible throughout the entire class and globally via `ClassName.var`.
Memory Location	Allocated on the **Thread Stack**.	Allocated on the **Heap** as part of the object.	Allocated in the **Method Area / Metaspace**.
Lifetime	Created on method entry, destroyed on exit.	Created when object instantiated, destroyed when GC reclaims object.	Created when class is loaded into JVM, destroyed when class is unloaded.
Default Values	No default values; must be initialized before use.	Default values assigned (0, 0.0, false, null).	Default values assigned (0, 0.0, false, null).

Static Members and Methods: Rules & Restrictions

- **Single Shared Copy:** Static members belong to the class rather than any individual instance. All instances share the exact same variable.
- **Invocation without Object:** Static methods can be called directly using class name: `ClassName.staticMethod()`.
- **Restrictions:**
  1. A static method can **only directly access static data** and call other static methods. It cannot access non-static instance fields directly.
  2. A static method **cannot use `this` or `super` keywords** because there is no current instance in context.

```
public class StaticDemo {
    int instanceCounter = 0;        // Increments per object
    static int globalCounter = 0;   // Shared across all objects

    StaticDemo() {
        instanceCounter++;
        globalCounter++;
    }

    static void displayGlobalCount() {
        // System.out.println(instanceCounter); // COMPILE ERROR: cannot make static reference
        System.out.println("Total Instances Created: " + globalCounter);
    }

    public static void main(String[] args) {
        new StaticDemo();
        new StaticDemo();
        new StaticDemo();
        StaticDemo.displayGlobalCount(); // Outputs: 3
    }
}
```

FREQUENCY: 4X (PYQ + QB) PYQ 2023, 2024, 2025, 2026 MARKS: 5 / 10 CO2 (BT6)

Q2.4: What is Constructor Overloading? Write a complete Java program to calculate Area using Constructor Overloading, and demonstrate Class Rectangle with Area and Perimeter.

Constructor: A constructor is a special member method invoked automatically when an object is instantiated using the `new` operator. It initializes the freshly allocated object state. A constructor has:

- The exact same name as the enclosing class.
- **No return type**, not even `void`.

Constructor Overloading: Defining multiple constructors within the same class, each having the same name but **different parameter lists** (different number, types, or order of parameters). It enables objects to be initialized in diverse configurations.

```
// Program: Area Calculation using Constructor Overloading and Class Rectangle
import java.util.Scanner;

class ShapeArea {
    double area;

    // Constructor 1: Default Constructor
    ShapeArea() {
        area = 0.0;
    }

    // Constructor 2: Overloaded for Square (one argument)
    ShapeArea(double side) {
        area = side * side;
        System.out.println("Square Area (side = " + side + ") : " + area);
    }

    // Constructor 3: Overloaded for Rectangle (two arguments)
    ShapeArea(double length, double breadth) {
        area = length * breadth;
        System.out.println("Rectangle Area (" + length + " x " + breadth + ") : " + area);
    }

    // Constructor 4: Overloaded for Circle (radius and boolean tag to differentiate)
    ShapeArea(double radius, boolean isCircle) {
        area = Math.PI * radius * radius;
        System.out.println("Circle Area (radius = " + radius + ") : " + area);
    }
}

// Dedicated Rectangle Class with Area and Perimeter (High-Frequency Exam Question)
class Rectangle {
    double length;
    double breadth;

    // Parameterized Constructor
    Rectangle(double l, double b) {
        length = l;
        breadth = b;
    }

    double calculateArea() {
        return length * breadth;
    }

    double calculatePerimeter() {
        return 2 * (length + breadth);
    }

    void display() {
        System.out.println("--- Rectangle Properties ---");
        System.out.println("Length    : " + length);
        System.out.println("Breadth   : " + breadth);
        System.out.println("Area      : " + calculateArea());
        System.out.println("Perimeter : " + calculatePerimeter());
    }
}
```

```
public class ConstructorOverloadingDemo {
    public static void main(String[] args) {
        System.out.println("=== PART 1: Constructor Overloading Demo ===");
        ShapeArea defaultShape = new ShapeArea();
        ShapeArea square = new ShapeArea(5.0);
        ShapeArea rect = new ShapeArea(6.0, 4.0);
        ShapeArea circle = new ShapeArea(3.0, true);

        System.out.println("\n=== PART 2: Rectangle Area & Perimeter with User Input ===");
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter length of rectangle: ");
        double len = sc.nextDouble();
        System.out.print("Enter breadth of rectangle: ");
        double br = sc.nextDouble();

        Rectangle userRect = new Rectangle(len, br);
        userRect.display();
    }
}
```

```
=== PART 1: Constructor Overloading Demo === Square Area (side = 5.0) : 25.0 Rectangle Area (6.0 x 4.0) : 24.0 Circle Area (radius = 3.0) : 28.274333882308138 === PART 2: Rectangle Area & Perimeter with User Input === Enter length of rectangle: 10 Enter breadth of rectangle: 5 --- Rectangle Properties --- Length : 10.0 Breadth : 5.0 Area : 50.0 Perimeter : 30.0
```

FREQUENCY: 4X (PYQ + QB) | PYQ 2023, 2024, 2025, 2026 | MARKS: 10 | CO2 (BT4)

Q2.5: Elaborate Argument Passing Mechanism in Java (Pass-by-Value vs. Pass-by-Reference). Illustrate with a program that passes Objects as Arguments and Returns an Object (Complex Number Addition).

A universal truth of the Java language is: **Java is STRICTLY Pass-by-Value**. There is NO pass-by-reference in Java in the C++ pointer sense. However, the behavior differs fundamentally based on whether the parameter is a primitive type or an object reference:

1. **Passing Primitive Data Types (int, double, char, etc.):** When a primitive variable is passed, a distinct bitwise copy of its value is pushed onto the called method's stack frame. Modifying the parameter inside the method has **zero effect** on the caller's variable in the invoking stack frame.
2. **Passing Object References:** When an object is passed, Java passes the **value of the reference address** (the 32-bit/64-bit pointer pointing to the heap). Both the caller's reference variable and the method's parameter variable point to the exact same object in the heap.
    - *Mutating Object State:* Modifying instance fields through the parameter reference (e.g., `param.real = 99`) **persists** and alters the object seen by the caller.
    - *Reassigning Reference:* Reassigning the parameter reference to point to a new object (e.g., `param = new Complex(0,0)`) only changes the local copy on the stack; the caller's reference continues pointing to the original heap object.
3. **Returning Objects from Methods:** A method can instantiate a new object on the heap and return its memory reference to the caller. This enables clean object transformation and functional method chaining.

```
// Program: Complex Number Addition Passing Objects as Arguments & Returning Object
class Complex {
    double real;
    double imag;

    // Parameterized Constructor
    Complex(double r, double i) {
        this.real = r;
        this.imag = i;
    }

    // Default Constructor
    Complex() {
        this(0.0, 0.0);
    }

    // Method accepting TWO Complex objects as arguments and returning a NEW Complex object
    static Complex add(Complex c1, Complex c2) {
        double rSum = c1.real + c2.real;
        double iSum = c1.imag + c2.imag;
        return new Complex(rSum, iSum); // Returning newly constructed object
    }

    // Instance method adding another Complex object to current object
    Complex add(Complex c2) {
        return new Complex(this.real + c2.real, this.imag + c2.imag);
    }

    void display(String label) {
        System.out.printf("%s: %.2f + %.2fi\n", label, real, imag);
    }
}

public class ArgumentPassingDemo {
    // Demonstrating primitive pass-by-value immutability
    static void modifyPrimitive(int val) {
        val = val + 100;
        System.out.println("Inside modifyPrimitive(): val = " + val);
    }

    public static void main(String[] args) {
        System.out.println("--- 1. Primitive Pass-by-Value Verification ---");
        int num = 50;
        System.out.println("Before call: num = " + num);
        modifyPrimitive(num);
        System.out.println("After call : num = " + num + " (Unchanged!)");

        System.out.println("\n--- 2. Object as Argument & Returning Object ---");
        Complex num1 = new Complex(3.5, 2.5);
        Complex num2 = new Complex(1.5, 4.5);

        num1.display("First Complex Number (C1)");
        num2.display("Second Complex Number (C2)");

        // Passing objects and receiving returned object
        Complex result = Complex.add(num1, num2);
        result.display("Resultant Sum (C1 + C2)");
```

```
    }
}
```

```
--- 1. Primitive Pass-by-Value Verification --- Before call: num = 50 Inside modifyPrimitive(): val = 150 After call : num = 50 (Unchanged!) --- 2. Object as Argument & Returning Object --- First Complex Number (C1): 3.50 + 2.50i Second Complex Number (C2): 1.50 + 4.50i Resultant Sum (C1 + C2): 5.00 + 7.00i
```

FREQUENCY: 3X (PYQ) PYQ 2023, 2024, 2026 MARKS: 5 CO2 (BT3)

Q2.6: Explain Recursion in Java. Write working programs for Fibonacci Series, Factorial, Permutations/Combinations (nPr, nCr), and Sum of First n Natural Numbers.

Recursion: The programming technique where a method calls itself directly or indirectly to solve a smaller subproblem of the original task. Every recursive method must possess two essential attributes:

1. **Base Case (Stopping Condition):** A condition under which the method returns a value without making further recursive calls, preventing infinite recursion and `StackOverflowError`.
2. **Recursive Step:** Calling the method with modified arguments that progressively move toward the base case.

```
// Program: Complete Recursion Toolkit (Fibonacci, Factorial, nPr, nCr, Natural Sum)
import java.util.Scanner;

public class RecursionExamToolkit {
    // 1. Factorial using Recursion: n! = n * (n-1)!
    static long factorial(int n) {
        if (n <= 1) return 1; // Base case
        return n * factorial(n - 1); // Recursive step
    }

    // 2. Fibonacci using Recursion: F(n) = F(n-1) + F(n-2)
    static int fibonacci(int n) {
        if (n <= 0) return 0; // Base case 1
        if (n == 1) return 1; // Base case 2
        return fibonacci(n - 1) + fibonacci(n - 2);
    }

    // 3. Permutations nPr = n! / (n - r)!
    static long nPr(int n, int r) {
        if (r > n) return 0;
        return factorial(n) / factorial(n - r);
    }

    // 4. Combinations nCr = n! / (r! * (n - r)!)
    static long nCr(int n, int r) {
        if (r > n) return 0;
        return factorial(n) / (factorial(r) * factorial(n - r));
    }

    // 5. Sum of first n natural numbers
    static int sumNatural(int n) {
        if (n <= 1) return n;
        return n + sumNatural(n - 1);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of terms for Fibonacci series: ");
        int terms = sc.nextInt();
        System.out.print("Fibonacci Series: ");
        for (int i = 0; i < terms; i++) {
            System.out.print(fibonacci(i) + " ");
        }
        System.out.println();

        System.out.print("\nEnter n and r for nPr and nCr calculations: ");
        int n = sc.nextInt();
        int r = sc.nextInt();
        System.out.println("Factorial(" + n + ") = " + factorial(n));
        System.out.println(n + "P" + r + " (Permutations)  = " + nPr(n, r));
        System.out.println(n + "C" + r + " (Combinations)  = " + nCr(n, r));
        System.out.println("Sum of first " + n + " natural numbers = " + sumNatural(n));
    }
}
```

```
Enter number of terms for Fibonacci series: 8 Fibonacci Series: 0 1 1 2 3 5 8 13 Enter n and r for nPr and nCr calculations: 5 2 Factorial(5) = 120 5P2 (Permutations) = 20 5C2 (Combinations) = 10 Sum of first 5 natural numbers = 15
```

FREQUENCY: 2X (PYQ) PYQ 2023 ADC303 Q3(B) & CECC303 Q2(D) MARKS: 5 / 10 CO2 (BT6)

Q2.7: Write a Java program to find the Square of an Integer number and the Square of a Float number using Method Overloading with user input.

Method Overloading: Occurs when multiple methods in the same class share the exact same name but have distinct parameter lists (different number of arguments, different data types, or different sequence of data types). Return types alone cannot distinguish overloaded methods.

```
// Program: Method Overloading for Square of Integer and Square of Float
import java.util.Scanner;

public class MethodOverloadingSquareDemo {
    // Overloaded Method 1: Computes square of an integer
    static int square(int num) {
        return num * num;
    }

    // Overloaded Method 2: Computes square of a float
    static float square(float num) {
        return num * num;
    }

    // Overloaded Method 3: Computes square of a double
    static double square(double num) {
        return num * num;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("=== Method Overloading Demonstration ===");

        System.out.print("Enter an integer: ");
        int intVal = sc.nextInt();
        System.out.println("Square of int " + intVal + " = " + square(intVal));

        System.out.print("\nEnter a floating-point number: ");
        float floatVal = sc.nextFloat();
        System.out.printf("Square of float %.2f = %.4f\n", floatVal, square(floatVal));

        sc.close();
    }
}
```

```
=== Method Overloading Demonstration === Enter an integer: 9 Square of int 9 = 81 Enter a floating-point number: 4.5 Square of float 4.50 = 20.2500
```

FREQUENCY: 1X (PYQ) PYQ 2023 ADC303 Q3(A) MARKS: 10 CO2 (BT3)

Q2.8: Write a Java program to read and display details of Employees using an Array of Objects with instance members: `Emp_ID`, `Emp_name`, `Emp_Salary`, parameterized constructor, and method `display()`.

In Java, an **Array of Objects** does not store actual object data contiguous in memory; rather, it allocates an array of *references* (pointers) on the heap. Each array slot must subsequently be instantiated individually with the `new` operator.

```
// Program: Array of Employee Objects with Parameterized Constructors
import java.util.Scanner;

class Employee {
    int Emp_ID;
    String Emp_name;
    double Emp_Salary;

    // Parameterized Constructor to initialize instance variables
    Employee(int id, String name, double salary) {
        this.Emp_ID = id;
        this.Emp_name = name;
        this.Emp_Salary = salary;
    }

    // Method to display information of an employee
    void display() {
        System.out.printf("%-10d %-25s %-12.2f\n", Emp_ID, Emp_name, Emp_Salary);
    }
}

public class EmployeeArrayDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("=== Employee Management System (Array of Objects) ===");

        System.out.print("Enter number of employees to register (e.g., 10): ");
        int n = sc.nextInt();
        sc.nextLine(); // Clear buffer

        Employee[] empList = new Employee[n]; // Array of object references

        for (int i = 0; i < n; i++) {
            System.out.println("\nEnter Details for Employee #" + (i + 1) + ":");
            System.out.print("Enter Employee ID: ");
            int id = sc.nextInt();
            sc.nextLine(); // Clear newline
            System.out.print("Enter Employee Name: ");
            String name = sc.nextLine();
            System.out.print("Enter Employee Salary: ");
            double salary = sc.nextDouble();

            empList[i] = new Employee(id, name, salary); // Dynamic heap instantiation
        }

        System.out.println("\n=================================================");
        System.out.printf("%-10s %-25s %-12s\n", "EMP ID", "EMPLOYEE NAME", "SALARY");
        System.out.println("=================================================");
        for (int i = 0; i < n; i++) {
            empList[i].display();
        }
        System.out.println("=================================================");
        sc.close();
    }
}
```

```
=== Employee Management System (Array of Objects) === Enter number of employees to register: 3 Enter Details for Employee #1: Enter Employee ID: 101 Enter Employee Name: Aditya Deshmukh Enter Employee Salary: 85000.00 Enter Details for Employee #2: Enter Employee ID: 102 Enter Employee Name: Priya Sharma Enter Employee Salary: 92000.00 Enter Details for Employee #3: Enter Employee ID: 103 Enter Employee Name: Rohan Patil Enter Employee Salary: 78000.00
================================================ EMP ID EMPLOYEE NAME SALARY
================================================ 101 Aditya Deshmukh 85000.00 102 Priya Sharma 92000.00 103 Rohan Patil 78000.00 ================================================
```

MODULE 3: ARRAYS, MATRICES & STRING HANDLING

FREQUENCY: 3X (PYQ) | PYQ 2023, 2024, 2025 | MARKS: 5 | CO3 (BT3)

Q3.1: Define Arrays in Java. Write a Java program to calculate the Sum and Average of Array Elements with user input.

Array: An array is a fixed-size, dynamically allocated data structure that stores a contiguous collection of homogeneous elements (elements of the same datatype). In Java:

- Arrays are treated as first-class **Objects** stored on the heap.
- Arrays have a built-in immutable `length` field reflecting their capacity.
- Indices are zero-based, running from `0` to `length - 1`. Accessing an illegal index throws `ArrayIndexOutOfBoundsException`.

```
// Program: Sum and Average of 1D Array with User Input
import java.util.Scanner;

public class ArraySumAverage {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter total number of elements in the array: ");
        int n = sc.nextInt();

        int[] arr = new int[n]; // Dynamic allocation on Heap
        int sum = 0;

        System.out.println("Enter " + n + " integer elements:");
        for (int i = 0; i < n; i++) {
            System.out.print("Element [" + i + "]: ");
            arr[i] = sc.nextInt();
            sum += arr[i]; // Accumulate sum
        }

        double average = (double) sum / n; // Explicit cast to preserve decimal precision

        System.out.println("\\n--- Statistical Summary ---");
        System.out.println("Sum of elements     : " + sum);
        System.out.printf("Average of elements : %.2f\\n", average);
    }
}
```

```
Enter total number of elements in the array: 5 Enter 5 integer elements: Element [0]: 14 Element [1]: 28 Element [2]: 35 Element [3]: 42 Element [4]: 56 --- Statistical Summary --- Sum of elements : 175 Average of elements : 35.00
```

FREQUENCY: 3X (PYQ + QB) PYQ 2024, 2025, 2026 MARKS: 5 / 10 CO3 (BT3)

Q3.2: Write a Java program to accept an m × n matrix and display it in natural form, and demonstrate Matrix Addition, Transpose, and Multiplication.

A 2D array in Java is implemented as an "array of arrays". An m × n matrix contains `m` rows and `n` columns.

- **Natural Form:** Displaying elements in row-major order with aligned tabular spacing and line breaks after each row.
- **Matrix Transpose:** Interchanging rows and columns such that element `A[i][j]` becomes A^T`[j][i]`. The dimensions transform from m × n to n × m.
- **Matrix Addition:** Element-wise sum `C[i][j] = A[i][j] + B[i][j]` (requires identical dimensions).
- **Matrix Multiplication:** Computes `C[i][j] = ∑ (A[i][k] × B[k][j])`. Permissible strictly when the number of columns in Matrix A equals the number of rows in Matrix B.

```
// Program: Complete Matrix Operations Suite (Natural Display, Transpose, Addition,
Multiplication)
import java.util.Scanner;

public class MatrixMasterSuite {
    // Helper to display a matrix in natural mathematical grid form
    static void displayMatrix(int[][] mat, String title) {
        System.out.println("\\n" + title + ":");
        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[0].length; j++) {
                System.out.printf("%6d", mat[i][j]);
            }
            System.out.println();
        }
    }

    // Matrix Multiplication Method: Computes product = A (m x k) * B (k x p)
    static int[][] multiply(int[][] a, int[][] b) {
        int r1 = a.length;
        int c1 = a[0].length;
        int r2 = b.length;
        int c2 = b[0].length;

        if (c1 != r2) {
            System.err.println("Multiplication Error: Columns of A (" + c1 + ") != Rows of B
(" + r2 + ")!");
            return null;
        }

        int[][] prod = new int[r1][c2];
        for (int i = 0; i < r1; i++) {
            for (int j = 0; j < c2; j++) {
                for (int k = 0; k < c1; k++) {
                    prod[i][j] += a[i][k] * b[k][j];
                }
            }
        }
        return prod;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("=== PART 1: Square Matrix Addition, Transpose & Multiplication
===");
        System.out.print("Enter matrix order N (for N x N matrices): ");
        int n = sc.nextInt();

        int[][] matA = new int[n][n];
        int[][] matB = new int[n][n];
        int[][] sumMat = new int[n][n];
        int[][] transposeA = new int[n][n];

        System.out.println("Enter elements of Matrix A (" + n + "x" + n + "):");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matA[i][j] = sc.nextInt();
                transposeA[j][i] = matA[i][j]; // Compute transpose simultaneously
```

```
            }
        }

        System.out.println("Enter elements of Matrix B (" + n + "x" + n + "):");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matB[i][j] = sc.nextInt();
                sumMat[i][j] = matA[i][j] + matB[i][j]; // Compute addition
            }
        }

        // Multiply A and B
        int[][] product = multiply(matA, matB);

        // Display results in natural grid form
        displayMatrix(matA, "Matrix A (Natural Form)");
        displayMatrix(matB, "Matrix B (Natural Form)");
        displayMatrix(transposeA, "Transpose of Matrix A (" + n + "x" + n + ")");
        displayMatrix(sumMat, "Matrix Sum (A + B)");
        if (product != null) {
            displayMatrix(product, "Matrix Product (A x B)");
        }
        sc.close();
    }
}
```

```
=== PART 1: Square Matrix Addition, Transpose & Multiplication === Enter matrix order N (for N x N matrices): 2 Enter elements of Matrix A (2x2): 1 2 3 4 Enter elements of Matrix B (2x2): 5 6 7 8 Matrix A (Natural Form): 1 2 3 4 Matrix B (Natural Form): 5 6 7 8 Transpose of Matrix A (2x2): 1 3 2 4 Matrix Sum (A + B): 6 8 10 12 Matrix Product (A x B): 19 22 43 50
```

FREQUENCY: 5X (PYQ + QB) | PYQ 2023, 2024, 2025, 2026 | MARKS: 5 | CO3 (BT2)

Q3.3: Elaborate the concept of String class and StringBuffer class. State their differences and describe any five methods of each class with examples. Justify how StringBuffer utilizes a better memory source than String.

1. The `String` Class (Immutable)

In Java, strings are objects of the `java.lang.String` class. **Strings are IMMUTABLE**: once a String object is created on the heap, its character sequence cannot be changed. Any operation that appears to modify a String (e.g., `concat()`, `toUpperCase()`) actually allocates a brand-new String object on the heap, leaving the original unaltered.

String Constant Pool (SCP): To conserve memory, the JVM maintains a special cache in the heap called the String Constant Pool. When a string literal is created (e.g., `String s1 = "Hello";`), JVM checks the pool first. If found, the existing reference is shared; if not, a new entry is added. If instantiated with `new String("Hello")`, two objects are created: one in normal heap memory and one in the pool.

2. The `StringBuffer` Class (Mutable & Thread-Safe)

The `StringBuffer` class represents a **MUTABLE**, growable sequence of characters. Characters and substrings can be inserted, appended, replaced, and deleted in place without creating extraneous intermediate objects.

Buffer Capacity: `StringBuffer` maintains an internal buffer capacity (default is 16 characters plus the initial string length). When the character sequence exceeds this capacity, the buffer automatically expands via formula:

```
New Capacity = (Old Capacity × 2) + 2
```

3. Memory Justification: Why StringBuffer is a Superior Memory Resource

UNIVERSITY EXAM PROOF: MEMORY COMPARISON

Consider concatenating 1,000 integers in a loop:

- **Using String (Heap Pollution):** `String s = ""; for(int i=0; i<1000; i++) s += i;`
  Because String is immutable, each concatenation creates a brand-new String object and copies the previous characters. This allocates **1,000 intermediate objects on the heap**, wasting Megabytes of memory and triggering aggressive Garbage Collection (GC) pauses.
- **Using StringBuffer (Zero Garbage):** `StringBuffer sb = new StringBuffer(); for(int i=0; i<1000; i++) sb.append(i);`
  StringBuffer mutates its internal `char[]` buffer in-place. It allocates **only one single object on the heap**, dynamically resizing only when the capacity threshold is reached. This results in O(1) amortized memory operations and near-zero garbage generation.

Comprehensive Comparison: String vs. StringBuffer

PARAMETER	STRING CLASS (`JAVA.LANG.STRING`)	STRINGBUFFER CLASS (`JAVA.LANG.STRINGBUFFER`)
Mutability	**Immutable** (state cannot be modified after instantiation).	**Mutable** (can modify character sequence in-place).
Storage Location	Stores literals in the **String Constant Pool (SCP)** or Heap.	Always allocated directly in regular **Heap Memory** (never pooled).
Performance & Speed	Slower for repeated modifications (creates excessive garbage objects).	Much faster for frequent concatenations and text modifications.
Memory Consumption	Higher when concatenating repeatedly due to multiple object allocations.	Highly memory efficient; reuses the existing allocated buffer.
Thread Safety	Thread-safe by nature because immutable objects cannot be corrupted.	Thread-safe; all core methods are **synchronized**.
Equality Operator (`equals()`)	Overrides `Object.equals()` to compare **content**.	Does NOT override `equals()`; compares **reference addresses**.

Top 5 Methods of String Class

1. `int length()`: Returns total count of Unicode characters. Example: `"RAIT".length()` → `4`.
2. `char charAt(int index)`: Returns the character at specified index. Example: `"Java".charAt(2)` → `'v'`.
3. `String substring(int beginIndex, int endIndex)`: Returns substring from begin to end-1. Example: `"Engineering".substring(0, 4)` → `"Engi"`.
4. `String toLowerCase() / toUpperCase()`: Converts all characters to lower/upper case. Example: `"mumbai".toUpperCase()` → `"MUMBAI"`.
5. `boolean equals(Object obj)`: Compares character contents for equality. Example: `"DYP".equals("DYP")` → `true`.

Top 5 Methods of StringBuffer Class

1. `StringBuffer append(String str)`: Appends the string to the end of the buffer. Example: `sb.append(" College")`.
2. `StringBuffer insert(int offset, String str)`: Inserts text at the specified index. Example: `sb.insert(4, " Patil")`.
3. `StringBuffer replace(int start, int end, String str)`: Replaces character range. Example: `sb.replace(0, 2, "XX")`.
4. `StringBuffer delete(int start, int end)`: Deletes characters from start to end-1. Example: `sb.delete(1, 3)`.
5. `StringBuffer reverse()`: Reverses the sequence of characters directly in-place. Example: `new StringBuffer("RADAR").reverse()`.

FREQUENCY: 3X (PYQ + QB) | PYQ 2023, 2024, 2026 | MARKS: 5 | CO3 (BT6)

Q3.4: Write a Java program to check whether a string is a Palindrome (traversing characters backwards and forwards) using StringBuffer.

```
// Program: String Palindrome Verification using StringBuffer and Manual Traversal
import java.util.Scanner;

public class PalindromeChecker {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string to test for palindrome: ");
        String original = sc.nextLine().trim();

        // --- METHOD 1: Using StringBuffer.reverse() ---
        StringBuffer sb = new StringBuffer(original);
        String reversed = sb.reverse().toString();

        if (original.equalsIgnoreCase(reversed)) {
            System.out.println("Result (StringBuffer): YES -> '" + original + "' is a
Palindrome!");
        } else {
            System.out.println("Result (StringBuffer): NO  -> '" + original + "' is NOT a
Palindrome.");
        }

        // --- METHOD 2: Character-by-character Two-Pointer Traversal ---
        boolean isPalin = true;
        int left = 0;
        int right = original.length() - 1;

        while (left < right) {
            if (Character.toLowerCase(original.charAt(left)) !=
Character.toLowerCase(original.charAt(right))) {
                isPalin = false;
                break;
            }
            left++;
            right--;
        }

        System.out.println("Result (Two-Pointer Verification): " + (isPalin ? "YES" : "NO"));
    }
}
```

```
Enter a string to test for palindrome: Madam Result (StringBuffer): YES -> 'Madam' is a Palindrome!
Result (Two-Pointer Verification): YES
```

MODULE 4: INHERITANCE, ABSTRACT CLASSES, INTERFACES & PACKAGES

FREQUENCY: 5X (PYQ + QB) | PYQ 2023, 2024, 2025, 2026 | MARKS: 5 / 8 | CO4 (BT2)

Q4.1: Discuss the importance of Inheritance in Java. List all types of Inheritance and explain why Java does not support Multiple Inheritance with classes.

Inheritance is a core mechanism in Object-Oriented Programming where a newly created class (derived / subclass / child) inherits fields and methods from an existing class (base / superclass / parent). It establishes an **IS-A** relationship (e.g., Dog IS-A Animal).

Importance and Benefits of Inheritance

1. **Code Reusability:** Subclasses can directly access and reuse pre-tested, bug-free methods of the superclass without redefining them.
2. **Extensibility:** New features can be added to existing systems by subclassing without touching or modifying the proven superclass source code.
3. **Runtime Polymorphism:** Method overriding enables a superclass reference variable to dynamically invoke subclass-specific implementations at runtime.
4. **Organized Hierarchical Structure:** Models real-world classifications naturally (e.g., Shape → Polygon → Rectangle).

Types of Inheritance in Java

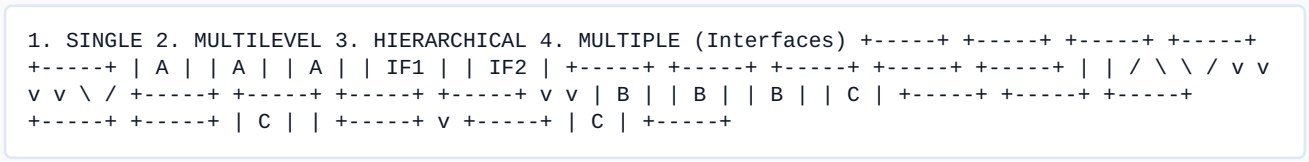


Figure 4.1: Classification of Inheritance Patterns in Java

1. **Single Inheritance:** A single subclass extends from a single superclass (A → B). Supported directly via `extends`.
2. **Multilevel Inheritance:** A class extends from a class that itself extends from another class (A → B → C). Supported directly.
3. **Hierarchical Inheritance:** Multiple subclasses extend from a common parent class (B ← A → C). Supported directly.
4. **Multiple Inheritance:** A subclass inherits from more than one superclass (A, B → C). **NOT supported with classes** in Java; achieved safely using **Interfaces**.
5. **Hybrid Inheritance:** A combination of two or more types of inheritance. Achieved in Java only through a combination of classes and interfaces.

Why Java Does NOT Support Multiple Inheritance with Classes (The Diamond Problem)

Consider Class A defining method `display()`. Both Class B and Class C extend Class A and override `display()`. If Class D were allowed to extend both B and C:

```
+----------------+ | Class A | display() +----------------+ / \ / \ +----------------+
+----------------+ | Class B | | Class C | | display() [B] | | display() [C] | +----------------+
+----------------+ \ / \ / +----------------+ | Class D | (Ambiguity: Which display() should D
inherit?) +----------------+
```

When an instance of D calls `d.display()`, the compiler cannot determine whether to execute Class B's or Class C's version. This ambiguity is known as the **Diamond Problem**. To eliminate complexity and keep the language clean, Java architects omitted multiple inheritance of state/classes. Because interfaces contain no instance state (variables are `public static final`), implementing multiple interfaces causes zero ambiguity.

FREQUENCY: 2X (PYQ) PYQ 2023, 2024 MARKS: 5 CO4 (BT6)

Q4.2: Define a superclass `Student` with instance variables `name`, `roll`, methods `read()`, `display()`, and a subclass `College` with instance variables `collegeName`, `code` with methods `get()`, `put()`. Create an object, invoke methods, and state the type of inheritance.

```
// Program: Single Inheritance Demonstration (Student -> College)
import java.util.Scanner;

// Superclass
class Student {
    String name;
    int roll;

    void read(Scanner sc) {
        System.out.print("Enter Student Name: ");
        name = sc.nextLine();
        System.out.print("Enter Roll Number: ");
        roll = sc.nextInt();
        sc.nextLine(); // Clear buffer
    }

    void display() {
        System.out.println("Student Name : " + name);
        System.out.println("Roll Number  : " + roll);
    }
}

// Subclass extending Student
class College extends Student {
    String collegeName;
    String code;

    void get(Scanner sc) {
        read(sc); // Reusing superclass method
        System.out.print("Enter College Name: ");
        collegeName = sc.nextLine();
        System.out.print("Enter College Code: ");
        code = sc.nextLine();
    }

    void put() {
        System.out.println("\\n--- Student & College Profile ---");
        display(); // Reusing superclass display
        System.out.println("College Name : " + collegeName);
        System.out.println("College Code : " + code);
    }
}

public class SingleInheritanceDemo {
    public static void main(String[] args) {
        System.out.println("Type of Inheritance: SINGLE INHERITANCE (College extends Student)\
\n");
        Scanner sc = new Scanner(System.in);
        College c = new College();
        c.get(sc);
        c.put();
    }
}
```

```
Type of Inheritance: SINGLE INHERITANCE (College extends Student) Enter Student Name: Rohan Kulkarni
Enter Roll Number: 42 Enter College Name: Ramrao Adik Institute of Technology (RAIT) Enter College
```

```
Code: 2413 --- Student & College Profile --- Student Name : Rohan Kulkarni Roll Number : 42 College Name : Ramrao Adik Institute of Technology (RAIT) College Code : 2413
```

FREQUENCY: 4X (PYQ + QB) PYQ 2023, 2024, 2025, 2026 MARKS: 10 CO4 (BT4)

Q4.3: Write a Java program to calculate the Volume of a Sphere using Multilevel Inheritance (Base accepts radius, Class 2 calculates area of circle, Class 3 calculates & displays volume of sphere).

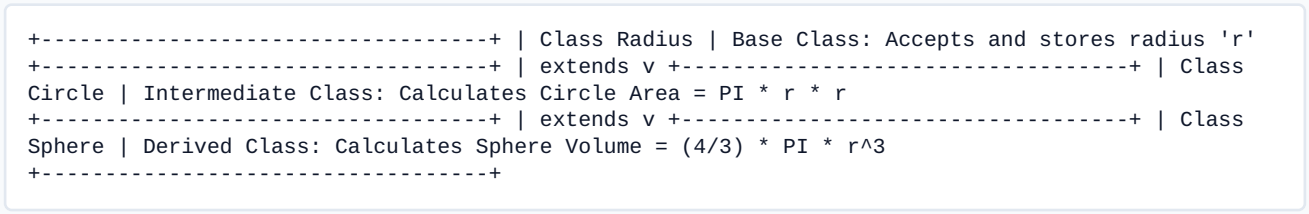


Figure 4.2: Multilevel Inheritance Chain for Sphere Calculation

```
// Program: Volume of Sphere using Multilevel Inheritance
import java.util.Scanner;

// Level 1: Base Class
class Radius {
    double r;

    void acceptRadius(Scanner sc) {
        System.out.print("Enter radius: ");
        r = sc.nextDouble();
    }
}

// Level 2: Intermediate Derived Class
class Circle extends Radius {
    double circleArea;

    double calculateArea() {
        circleArea = Math.PI * r * r;
        return circleArea;
    }
}

// Level 3: Final Derived Class
class Sphere extends Circle {
    double sphereVolume;

    void calculateAndDisplayVolume() {
        // Calculate circle area first from parent class
        calculateArea();

        // Volume of sphere = (4/3) * PI * r^3 = (4/3) * r * CircleArea
        sphereVolume = (4.0 / 3.0) * Math.PI * Math.pow(r, 3);

        System.out.println("\\n--- Multilevel Inheritance Calculation Results ---");
        System.out.printf("Radius (r)         : %.2f units\\n", r);
        System.out.printf("Circle Base Area   : %.4f sq units\\n", circleArea);
        System.out.printf("Volume of Sphere   : %.4f cubic units\\n", sphereVolume);
    }
}

public class MultilevelSphereDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Sphere sphereObj = new Sphere();
        sphereObj.acceptRadius(sc);
        sphereObj.calculateAndDisplayVolume();
    }
}
```

```
Enter radius: 7.5 --- Multilevel Inheritance Calculation Results --- Radius (r) : 7.50 units Circle Base Area : 176.7146 sq units Volume of Sphere : 1767.1459 cubic units
```

FREQUENCY: 3X (PYQ + QB) PYQ 2024, 2025, 2026 MARKS: 5 CO4 (BT2)

Q4.4: Describe the three usages of the `super` keyword and the three usages of the `final` keyword in Java with working code.

The `super` Keyword (3 Core Usages)

1. **`super.variableName`:** Used to access hidden fields of the immediate superclass when shadowed by subclass fields.
2. **`super.methodName()`:** Used to invoke the immediate superclass method that has been overridden by the subclass.
3. **`super(arguments)`:** Used to invoke the immediate superclass constructor. Must be the *very first statement* inside the subclass constructor.

The `final` Keyword (3 Core Usages)

1. **Final Variable:** Creates a constant whose value *cannot be reassigned* after initialization.
2. **Final Method:** Prevents *method overriding* by any subclass.
3. **Final Class:** Prevents *class inheritance* (the class cannot be extended, e.g., `java.lang.String`).

```
// Program: Demonstrating All Usages of 'super' and 'final'
class Parent {
    int value = 100;

    Parent(String message) {
        System.out.println("Parent Constructor: " + message);
    }

    void show() {
        System.out.println("Parent show() method");
    }

    final void secureToken() {
        System.out.println("Final Method: Subclasses CANNOT override this!");
    }
}

class Child extends Parent {
    int value = 200; // Shadowing parent field

    Child() {
        super("Invoking Parent Constructor via super()"); // 1. super constructor call
    }

    void test() {
        System.out.println("Subclass value: " + value);
        System.out.println("Parent value  : " + super.value); // 2. Accessing hidden parent
variable
        super.show(); // 3. Invoking overridden parent method
    }

    // void secureToken() {} // COMPILE ERROR: cannot override final method from Parent
}

final class ImmutableClass {
    final double PI = 3.14159; // Final variable: constant
}
// class Subclass extends ImmutableClass {} // COMPILE ERROR: cannot inherit from final class
```

FREQUENCY: 3X (PYQ + QB) | PYQ 2023, 2024, 2025 | MARKS: 5 | CO4 (BT4)

Q4.5: Compare and contrast Method Overloading and Method Overriding in Java in detail.

PARAMETER	METHOD OVERLOADING	METHOD OVERRIDING
Definition	Multiple methods in the same class sharing the same name but different signatures.	Subclass providing a specific implementation of a method already defined in its superclass.
Polymorphism Type	**Compile-time (Static) Polymorphism**.	**Run-time (Dynamic) Polymorphism**.
Binding Time	Early Binding (resolved by compiler at compile-time).	Late Binding (resolved dynamically by JVM at runtime based on object).
Method Signature	**MUST be different** (different number, types, or order of parameters).	**MUST be identical** (exact same method name, parameter types, and count).
Return Type	Can be the same or different (return type alone cannot overload).	Must be identical or covariant (subclass of the superclass return type).
Inheritance Requirement	Can occur within a single class without any inheritance.	Requires at least two classes in an **IS-A (Inheritance)** relationship.
Private / Static Methods	Private and static methods can be overloaded.	Private and static methods **CANNOT** be overridden (static methods are hidden).
Access Modifiers	Can have any access modifier.	Subclass method cannot assign weaker access privileges (e.g., cannot reduce public to protected).

FREQUENCY: 2X (PYQ) | PYQ 2024, 2026 | MARKS: 10 | CO4 (BT3)

Q4.6: Differentiate between Abstract Class and Interface. Write a Java program which has an abstract class `Solid` and implements `Cylinder` and `Sphere` to find their volumes.

FEATURE	ABSTRACT CLASS	INTERFACE
Method Types	Can contain both abstract (without body) and non-abstract concrete methods.	Contains only abstract methods (except default and static methods in Java 8+).
Fields / Variables	Can have instance variables, static, final, or non-final variables with any access modifier.	All variables are implicitly `public static final` (constants).
Constructors	Can define constructors (invoked when subclass is instantiated via `super()`).	**Cannot have constructors** (interfaces cannot be instantiated).
Multiple Inheritance	A class can extend only **one** abstract class (Single inheritance).	A class can implement **multiple** interfaces (Multiple inheritance).
Keyword	Declared using `abstract class`; inherited using `extends`.	Declared using `interface`; implemented using `implements`.
Speed / Performance	Slightly faster since method lookup is direct via vtable.	Slightly slower due to dynamic interface table (itable) search.

```
// Program: Abstract Class Solid with Cylinder and Sphere Subclasses
abstract class Solid {
    double radius;

    Solid(double r) {
        this.radius = r;
    }

    // Abstract method: must be overridden by all concrete derived solids
    abstract double calculateVolume();
    abstract double calculateSurfaceArea();

    void displayInfo(String solidName) {
        System.out.println("\\n--- Solid: " + solidName + " ---");
        System.out.printf("Radius        : %.2f units\\n", radius);
        System.out.printf("Volume        : %.4f cubic units\\n", calculateVolume());
        System.out.printf("Surface Area  : %.4f sq units\\n", calculateSurfaceArea());
    }
}

class SphereSolid extends Solid {
    SphereSolid(double r) {
        super(r);
    }

    double calculateVolume() {
        return (4.0 / 3.0) * Math.PI * Math.pow(radius, 3);
    }

    double calculateSurfaceArea() {
        return 4.0 * Math.PI * Math.pow(radius, 2);
    }
}

class CylinderSolid extends Solid {
    double height;

    CylinderSolid(double r, double h) {
        super(r);
        this.height = h;
    }

    double calculateVolume() {
        return Math.PI * Math.pow(radius, 2) * height;
    }

    double calculateSurfaceArea() {
        return (2 * Math.PI * radius * height) + (2 * Math.PI * Math.pow(radius, 2));
    }
}

class ConeSolid extends Solid {
    double height;

    ConeSolid(double r, double h) {
        super(r);
        this.height = h;
    }
```

```
    double calculateVolume() {
        return (1.0 / 3.0) * Math.PI * Math.pow(radius, 2) * height;
    }

    double calculateSurfaceArea() {
        double slantHeight = Math.sqrt(radius * radius + height * height);
        return Math.PI * radius * (radius + slantHeight);
    }
}

public class AbstractSolidDemo {
    public static void main(String[] args) {
        Solid sphere = new SphereSolid(5.0);
        sphere.displayInfo("SPHERE");

        Solid cylinder = new CylinderSolid(3.0, 10.0);
        cylinder.displayInfo("CYLINDER");

        Solid cone = new ConeSolid(3.0, 4.0);
        cone.displayInfo("CONE");
    }
}
```

FREQUENCY: 4X (PYQ + QB) PYQ 2023, 2024, 2025, 2026 MARKS: 10 CO4 (BT6)

Q4.7: High-Yield Exam Design Problem: Consider a university where students who participate in National Games / Olympics receive fixed grace marks. Design an application using Interface and Base Class that keeps Academic Marks and Sports Grace Marks separate, generates total marks, and invokes methods using references.

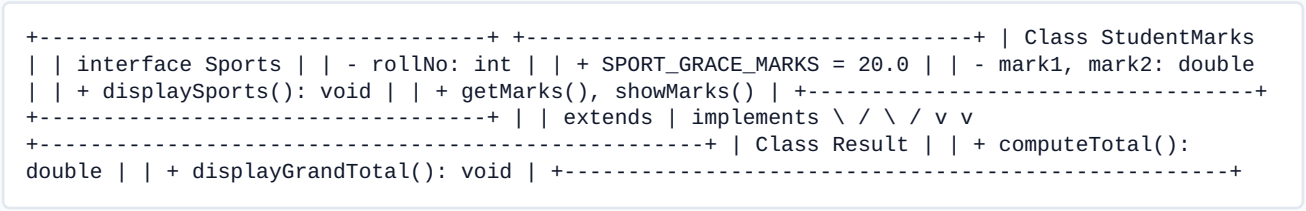


Figure 4.3: Multiple Inheritance via Interface and Base Class for University Grading

```
// Program: University Sports Grace Marks System (Interface & Base Class)
import java.util.Scanner;

// Base Class: Stores Student Academic Details
class StudentMarks {
    int rollNo;
    String name;
    double academicMark1;
    double academicMark2;

    void getAcademicDetails(Scanner sc) {
        System.out.print("Enter Student Roll Number: ");
        rollNo = sc.nextInt();
        sc.nextLine(); // Clear newline
        System.out.print("Enter Student Name: ");
        name = sc.nextLine();
        System.out.print("Enter Marks for Subject 1: ");
        academicMark1 = sc.nextDouble();
        System.out.print("Enter Marks for Subject 2: ");
        academicMark2 = sc.nextDouble();
    }

    void displayAcademicDetails() {
        System.out.println("Roll Number      : " + rollNo);
        System.out.println("Student Name     : " + name);
        System.out.println("Subject 1 Marks  : " + academicMark1);
        System.out.println("Subject 2 Marks  : " + academicMark2);
        System.out.println("Academic Subtotal: " + (academicMark1 + academicMark2));
    }
}

// Interface: Defines Fixed Sports Grace Marks
interface Sports {
    double SPORT_GRACE_MARKS = 25.0; // Implicitly public static final

    void displaySportsMarks(); // Implicitly public abstract
}

// Derived Class: Inherits StudentMarks and Implements Sports
class Result extends StudentMarks implements Sports {
    double grandTotal;

    // Implementation of Interface method
    public void displaySportsMarks() {
        System.out.println("Sports Grace Mark: " + SPORT_GRACE_MARKS + " (Awarded for National
Sports)");
    }

    void computeAndDisplayResult() {
        double academicTotal = academicMark1 + academicMark2;
        grandTotal = academicTotal + SPORT_GRACE_MARKS;

        System.out.println("\\n=========================================");
        System.out.println("       DY PATIL UNIVERSITY EXAM REPORT     ");
        System.out.println("=========================================");
        displayAcademicDetails(); // Base class method
        displaySportsMarks();     // Interface method
        System.out.println("-----------------------------------------");
```

```
        System.out.println("GRAND TOTAL MARKS: " + grandTotal + " / 225.0");
        System.out.println("Final Result     : PASSED WITH SPORTS DISTINCTION");
        System.out.println("==========================================");
    }
}

public class UniversitySportsExamDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Result studentResult = new Result();

        studentResult.getAcademicDetails(sc);

        // Invoking via Base Class Reference
        StudentMarks baseRef = studentResult;
        System.out.println("\\n>>> Verification: Invoking via Base Class Reference:");
        baseRef.displayAcademicDetails();

        // Invoking via Interface Reference
        Sports sportsRef = studentResult;
        System.out.println("\\n>>> Verification: Invoking via Interface Reference:");
        sportsRef.displaySportsMarks();

        // Final Complete Report
        studentResult.computeAndDisplayResult();
    }
}
```

FREQUENCY: 2X (PYQ + QB) PYQ 2024, 2026 MARKS: 5 CO4 (BT2)

Q4.8: What is a Package in Java? Explain the types of packages, how to create and compile user-defined packages, and the role of CLASSPATH.

Package: A package is a grouped namespace that encapsulates related classes, interfaces, and sub-packages.

Types of Packages

1. **Built-in Packages:** Shipped as part of the Java Standard Edition API:
    - `java.lang` : Core language classes (Math, String, Thread, Object) - automatically imported.
    - `java.util` : Utility frameworks, collections, Scanner, Date, Random.
    - `java.io` : Data stream reading and writing (File, BufferedReader).
    - `java.awt` & `javax.swing` : Graphical User Interface toolkits.
2. **User-Defined Packages:** Packages developed by software engineers to organize application modules and prevent naming collisions.

Creation, Compilation, and Execution of User-Defined Packages

1. **Declare Package:** Add `package packageName;` as the *first statement* in the source file:

```
// File: MyMath.java
package dypatil.mathutil;

public class MyMath {
    public static int square(int x) { return x * x; }
}
```

2. **Compile with `-d` Flag:** The `-d .` option instructs `javac` to create matching subdirectories automatically:

```
javac -d . MyMath.java
```

   This generates directory structure `dypatil/mathutil/MyMath.class`.

3. **Import and Use in Client Application:**

```
import dypatil.mathutil.MyMath;

public class PackageTester {
    public static void main(String[] args) {
        System.out.println("Square of 9: " + MyMath.square(9));
    }
}
```

4. **Role of `CLASSPATH`:** Environment variable that tells the JVM where to look for user-defined `.class` files and external libraries. Syntax: `export CLASSPATH=.:/path/to/classes`.

FREQUENCY: 1X (PYQ) PYQ 2024 CAC303 Q3(A) MARKS: 10 CO4 (BT4)

Q4.9: Write a program using Multilevel Inheritance: Define class A with two integer variables and constructor; define class B extending A with variable C and constructor; define class C extending B with method to calculate and display sum and product of all variables.

```
// Program: Multilevel Inheritance with Parameterized Constructors (A -> B -> C)
import java.util.Scanner;

// Level 1: Superclass A
class AlphaA {
    int x, y;

    AlphaA(int a, int b) {
        this.x = a;
        this.y = b;
    }

    void displayA() {
        System.out.println("Class A -> x = " + x + ", y = " + y);
    }
}

// Level 2: Intermediate Subclass B extending A
class BetaB extends AlphaA {
    int z;

    BetaB(int a, int b, int c) {
        super(a, b); // Explicitly invoke Superclass A constructor
        this.z = c;
    }

    void displayB() {
        displayA();
        System.out.println("Class B -> z = " + z);
    }
}

// Level 3: Final Subclass C extending B
class GammaC extends BetaB {
    GammaC(int a, int b, int c) {
        super(a, b, c); // Pass parameters up the multilevel chain
    }

    void calculateAndDisplay() {
        displayB();
        int sum = x + y + z;
        long product = (long) x * y * z;
        System.out.println("\\n--- Calculations across Multilevel Hierarchy ---");
        System.out.println("Sum (x + y + z)     : " + sum);
        System.out.println("Product (x * y * z) : " + product);
    }
}

public class MultilevelVariableDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("=== Multilevel Inheritance Variable Aggregation (A -> B -> C)
===");
        System.out.print("Enter three integers (x, y, z): ");
        int a = sc.nextInt();
        int b = sc.nextInt();
        int c = sc.nextInt();
```

```
        GammaC obj = new GammaC(a, b, c);
        obj.calculateAndDisplay();
        sc.close();
    }
}
```

```
=== Multilevel Inheritance Variable Aggregation (A -> B -> C) === Enter three integers (x, y, z): 10
20 30 Class A -> x = 10, y = 20 Class B -> z = 30 --- Calculations across Multilevel Hierarchy ---
Sum (x + y + z) : 60 Product (x * y * z) : 6000
```

FREQUENCY: 2X (PYQ) | PYQ 2023 CECC303 Q2(C) & 2024 CAC303 | MARKS: 5 | CO4 (BT3)

Q4.10: Write a program to display Today's Date by using `java.util` package (`Date` and `Calendar` classes).

```
// Program: Display Today's Date using java.util Package
import java.util.Date;
import java.text.SimpleDateFormat;
import java.util.Calendar;

public class DisplayCurrentDateDemo {
    public static void main(String[] args) {
        // 1. Using java.util.Date
        Date today = new Date();
        System.out.println("Current Date & Time (Raw java.util.Date): " + today);

        // 2. Formatting using java.text.SimpleDateFormat
        SimpleDateFormat sdfStandard = new SimpleDateFormat("dd-MM-yyyy");
        SimpleDateFormat sdfDetailed = new SimpleDateFormat("EEEE, MMMM dd, yyyy 'at' hh:mm:ss
a");

        System.out.println("Formatted Date (DD-MM-YYYY)            : " +
sdfStandard.format(today));
        System.out.println("Detailed Date & Time                   : " +
sdfDetailed.format(today));

        // 3. Using java.util.Calendar to extract individual fields
        Calendar cal = Calendar.getInstance();
        System.out.println("\\n--- Calendar Breakdown (java.util.Calendar) ---");
        System.out.println("Day of Month : " + cal.get(Calendar.DAY_OF_MONTH));
        System.out.println("Month        : " + (cal.get(Calendar.MONTH) + 1)); // Note: Month
is 0-indexed
        System.out.println("Year         : " + cal.get(Calendar.YEAR));
    }
}
```

```
Current Date & Time (Raw java.util.Date): Sun Oct 20 16:45:12 IST 2026 Formatted Date (DD-MM-YYYY) :
20-10-2026 Detailed Date & Time : Sunday, October 20, 2026 at 04:45:12 PM --- Calendar Breakdown
(java.util.Calendar) --- Day of Month : 20 Month : 10 Year : 2026
```

MODULE 5: EXCEPTION HANDLING & MULTITHREADING

FREQUENCY: 3X (PYQ + QB) | PYQ 2024, 2026 | MARKS: 5 | CO5 (BT2)

Q5.1: Differentiate between Errors and Exceptions in Java. Explain the Exception Hierarchy, Checked vs. Unchecked Exceptions, and core keywords (try, catch, finally, throw, throws).

Errors vs. Exceptions in Java

PARAMETER	ERROR (`JAVA.LANG.ERROR`)	EXCEPTION (`JAVA.LANG.EXCEPTION`)
Nature	Indicates catastrophic, irrecoverable system-level failures outside application control.	Indicates exceptional conditions that a reasonable application might want to catch and handle.
Origin / Cause	Caused by environment, hardware, or JVM failures (e.g., memory exhaustion).	Caused by programmatic flaws, invalid user input, or missing resources.
Recoverability	**Non-recoverable**. Application cannot gracefully recover; JVM terminates.	**Recoverable**. Can be caught and handled gracefully using `try-catch`.
Type	Belongs to **Unchecked** category.	Includes both **Checked** (compile-time) and **Unchecked** (runtime) types.
Examples	`OutOfMemoryError`, `StackOverflowError`, `VirtualMachineError`.	`ArithmeticException`, `NullPointerException`, `IOException`, `SQLException`.

Java Exception Hierarchy

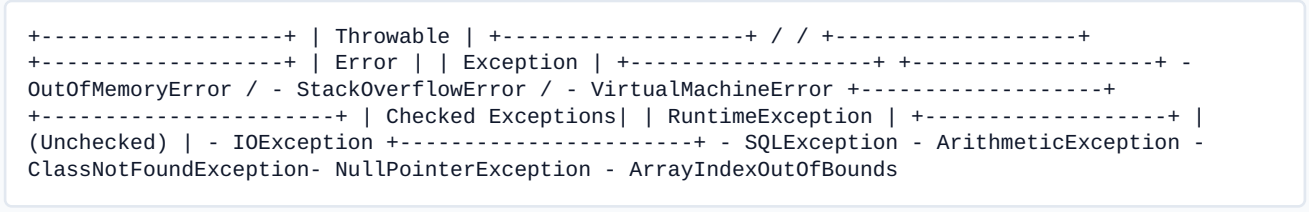


Figure 5.1: The Java Throwable Class Hierarchy

- **Checked Exceptions:** Checked at compile-time by the Java compiler. If a method calls code that throws a checked exception, it *must* either catch it or declare it via `throws`. Examples: `IOException`, `SQLException`.

- **Unchecked Exceptions (Runtime Exceptions):** Subclasses of `RuntimeException`. Not verified at compile-time; usually reflect logical programming bugs. Examples: `ArithmeticException`, `NullPointerException`.

The 5 Core Keywords

- `try`: Encloses the block of code where exceptions may potentially be thrown.
- `catch`: Encloses the handler block that catches and handles a specific exception type thrown by the corresponding `try` block.
- `finally`: Encloses cleanup code guaranteed to execute regardless of whether an exception was thrown, caught, or missed (even executes before a `return` statement).
- `throw`: Explicitly throws a specific exception instance from within a method body: `throw new MyException("Error message");`.
- `throws`: Appended to a method signature to declare that the method may pass specific checked exceptions to its caller.

FREQUENCY: 6X (PYQ + QB) | PYQ 2023, 2024, 2025, 2026 | MARKS: 5 | CO5 (BT3)

Q5.2: Write a Java program to accept two integers from the user, perform division, and demonstrate the handling of Exception (Divide by Zero) using try-catch blocks. Explain Multiple Catch blocks.

```
// Program: Divide by Zero Exception Handling and Multiple Catch Blocks
import java.util.Scanner;
import java.util.InputMismatchException;

public class DivisionExceptionDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.println("--- Integer Division Safe Calculator ---");
            System.out.print("Enter Numerator (integer a): ");
            int a = sc.nextInt();

            System.out.print("Enter Denominator (integer b): ");
            int b = sc.nextInt();

            // Guarded operation: may throw ArithmeticException if b == 0
            int result = a / b;

            System.out.println("SUCCESS: " + a + " / " + b + " = " + result);

            // Array index testing for multiple catch demonstration
            int[] testArr = {10, 20, 30};
            System.out.println("Accessing testArr[2]: " + testArr[2]);

        } catch (ArithmeticException e) {
            // Specific Handler for Division by Zero
            System.err.println("\n[CAUGHT ARITHMETIC EXCEPTION]: Division by zero is
mathematically undefined!");
            System.err.println("Exception message: " + e.getMessage());

        } catch (InputMismatchException e) {
            // Specific Handler for non-numeric input
            System.err.println("\n[CAUGHT INPUT MISMATCH]: You must enter integer numbers
only!");

        } catch (ArrayIndexOutOfBoundsException e) {
            // Specific Handler for Array Index Bounds
            System.err.println("\n[CAUGHT ARRAY INDEX EXCEPTION]: Illegal array index
accessed!");

        } catch (Exception e) {
            // Generic Fallback Handler: MUST BE PLACED LAST
            System.err.println("\n[GENERIC EXCEPTION HANDLER]: An unexpected error occurred: "
+ e);

        } finally {
            // Guaranteed Cleanup Block
            System.out.println("\n[FINALLY BLOCK EXECUTED]: Releasing resources and closing
scanner.");
            sc.close();
        }

        System.out.println("Program continues normal execution after try-catch-finally
block.");
    }
}
```

```
--- Execution Run 1: Invalid Denominator (Divide by Zero) --- Enter Numerator (integer a): 50 Enter Denominator (integer b): 0 [CAUGHT ARITHMETIC EXCEPTION]: Division by zero is mathematically undefined! Exception message: / by zero [FINALLY BLOCK EXECUTED]: Releasing resources and closing scanner. Program continues normal execution after try-catch-finally block.
```

CRITICAL EXAM RULE: MULTIPLE CATCH ORDER

When using multiple catch blocks, **subclass exceptions MUST be placed before superclass exceptions**. For example, `catch (ArithmeticException e)` must precede `catch (Exception e)`. If `catch (Exception e)` is placed first, the compiler rejects the code with an *"unreachable catch block"* compilation error.

FREQUENCY: 2X (PYQ + QB) | PYQ 2023, 2025 | MARKS: 10 | CO5 (BT6)

Q5.3: High-Yield Exam Design Problem: Explain Custom / User-Defined Exceptions. Write a Java program to calculate the Square of an Even Number provided as input by the user; if an Odd number is entered, throw an explicit custom exception (`OddNumberException`).

User-Defined / Custom Exceptions: While Java provides dozens of built-in exception classes, enterprise applications frequently require application-specific business logic errors. A custom exception is created by defining a class that extends `java.lang.Exception` (for checked custom exceptions) or `java.lang.RuntimeException` (for unchecked custom exceptions).

Steps to Implement Custom Exception

1. Create a class extending `Exception`.
2. Define constructors (default and parameterized accepting an error string) that call `super(message)`.
3. Use the `throw` keyword inside conditional validation logic to fire the exception.
4. Declare the exception in the method signature using the `throws` keyword.
5. Enclose the invocation in a `try-catch` block to catch and handle the custom error.

```
// Program: Custom User-Defined Exception (OddNumberException)
import java.util.Scanner;

// Step 1 & 2: Define Custom Exception Class
class OddNumberException extends Exception {
    public OddNumberException(String message) {
        super(message); // Pass custom message to parent Exception class
    }
}

public class CustomExceptionSquareDemo {
    // Method that enforces validation and calculates square
    static int calculateSquareOfEven(int num) throws OddNumberException {
        if (num % 2 != 0) {
            // Step 3: Explicitly throw custom exception if number is odd
            throw new OddNumberException("Validation Error: Input number (" + num + ") is ODD!
Only EVEN numbers are permitted.");
        }
        return num * num; // Executed only if number is even
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("=== Even Number Squaring Application ===");
        System.out.print("Please enter an integer: ");

        try {
            int input = sc.nextInt();
            int result = calculateSquareOfEven(input);
            System.out.println("SUCCESS: The square of even number " + input + " is: " +
result);

        } catch (OddNumberException e) {
            // Handling the custom exception gracefully
            System.err.println("\n>>> [CUSTOM EXCEPTION CAUGHT] <<<");
            System.err.println(e.getMessage());
            System.err.println("Action: Please re-run the application and supply an even
integer (e.g., 2, 4, 6, 8...).");

        } catch (Exception e) {
            System.err.println("General Input Error: " + e.getMessage());

        } finally {
            System.out.println("\n[Process Completed]: Thank you for using the application.");
            sc.close();
        }
    }
}
```

```
=== Even Number Squaring Application === Please enter an integer: 7 >>> [CUSTOM EXCEPTION CAUGHT] <<<
Validation Error: Input number (7) is ODD! Only EVEN numbers are permitted. Action: Please re-run the
application and supply an even integer (e.g., 2, 4, 6, 8...). [Process Completed]: Thank you for
using the application.
```

FREQUENCY: 4X (PYQ + QB) PYQ 2023, 2024, 2025, 2026 MARKS: 5 / 10 CO5 (BT2)

Q5.4: Illustrate the complete Life Cycle of a Thread in Java with a neat state transition diagram. Explain Thread creation: extending `Thread` class vs. implementing `Runnable` interface.

Multithreading: A concurrent execution mechanism where multiple lightweight sub-processes (threads) run concurrently within a single program, sharing the same process memory space and maximizing CPU efficiency.

Thread Life Cycle: 5 Major States

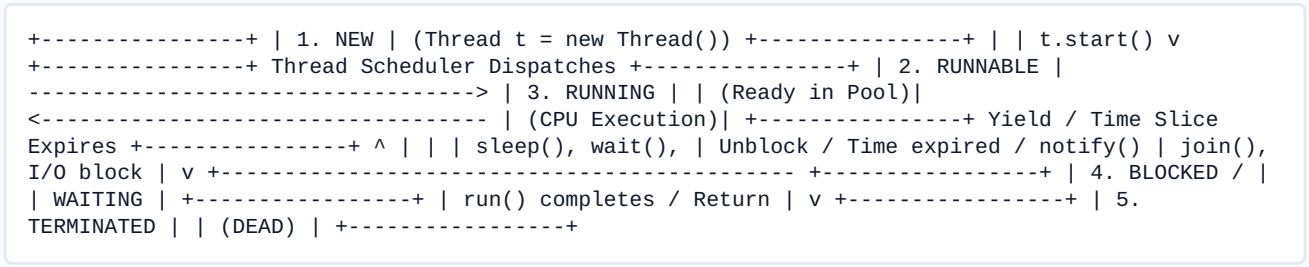


Figure 5.2: Complete State Transition Diagram of a Java Thread

1. **New (Born) State:** A thread object is created (`Thread t = new MyThread();`), but `start()` has not yet been invoked. It is not yet eligible to run.
2. **Runnable State:** After `t.start()` is called, the thread moves to the Runnable pool. It is ready for execution, waiting for the Thread Scheduler to assign CPU time.
3. **Running State:** The thread scheduler selects the thread from the runnable pool and allocates a CPU core. The code inside its `run()` method is actively executing.
4. **Blocked / Waiting / Timed Waiting State:** The thread is temporarily suspended:
    - Sleeping via `Thread.sleep(millis)`.
    - Waiting for a lock / synchronized monitor.
    - Waiting for another thread via `wait()` or `join()`.
    - Waiting for I/O operations to complete.

    Once the condition is resolved, it returns to the **Runnable** state (never directly to Running).
5. **Terminated (Dead) State:** The thread's `run()` method reaches completion, or an uncaught exception terminates it. A dead thread cannot be restarted.

Thread Creation: `Thread` Class vs. `Runnable` Interface

PARAMETER	EXTENDING `java.lang.Thread` CLASS	IMPLEMENTING `java.lang.Runnable` INTERFACE
Inheritance Flexibility	Because Java does not support multiple inheritance, a class extending `Thread` **cannot extend any other class**.	Highly flexible! The class implements `Runnable` while remaining free to **extend another superclass**.
Separation of Concerns	Tightly couples the task logic with the thread control mechanism.	Cleanly separates the unit of work (task) from the thread execution runner.
Thread Pool Support	Inefficient; each task is tied to a full Thread object.	Supported by standard Java concurrency frameworks (`ExecutorService`).
Industry Recommendation	Use only when overriding other methods of the Thread class.	**Strongly Recommended** as the standard, flexible approach.

FREQUENCY: 4X (PYQ + QB) | PYQ 2023, 2024, 2025, 2026 | MARKS: 10 | CO5 (BT3)

Q5.5: Write a Multithreaded Java program where One Thread generates and prints Odd Numbers and another Thread generates and prints Even Numbers concurrently.

```
// Program: Concurrent Odd and Even Number Printers using Multithreading
// Thread 1: Generates Even Numbers by extending Thread class
class EvenNumberThread extends Thread {
    int maxLimit;

    EvenNumberThread(int limit) {
        this.maxLimit = limit;
        setName("Even-Thread"); // Assign meaningful thread name
    }

    public void run() {
        System.out.println("Starting: " + getName());
        for (int i = 2; i <= maxLimit; i += 2) {
            System.out.println("[" + getName() + "] -> Even: " + i);
            try {
                Thread.sleep(100); // Simulate processing delay (100ms)
            } catch (InterruptedException e) {
                System.err.println(getName() + " interrupted!");
            }
        }
        System.out.println("Finished: " + getName());
    }
}

// Thread 2: Generates Odd Numbers by implementing Runnable interface
class OddNumberRunnable implements Runnable {
    int maxLimit;

    OddNumberRunnable(int limit) {
        this.maxLimit = limit;
    }

    public void run() {
        String threadName = Thread.currentThread().getName();
        System.out.println("Starting: " + threadName);
        for (int i = 1; i <= maxLimit; i += 2) {
            System.out.println("[" + threadName + "] -> Odd : " + i);
            try {
                Thread.sleep(100); // Simulate processing delay (100ms)
            } catch (InterruptedException e) {
                System.err.println(threadName + " interrupted!");
            }
        }
        System.out.println("Finished: " + threadName);
    }
}

public class OddEvenMultithreadingDemo {
    public static void main(String[] args) {
        int limit = 10;
        System.out.println("=== Multithreading Concurrent Odd/Even Demonstration ===");

        // Instantiating Thread 1 (Extending Thread)
        EvenNumberThread evenThread = new EvenNumberThread(limit);

        // Instantiating Thread 2 (Implementing Runnable)
        OddNumberRunnable oddRunnable = new OddNumberRunnable(limit);
        Thread oddThread = new Thread(oddRunnable, "Odd-Thread");
```

```
        // Set thread priorities (1 to 10)
        evenThread.setPriority(Thread.NORM_PRIORITY);
        oddThread.setPriority(Thread.NORM_PRIORITY);

        // Transition from NEW to RUNNABLE state
        evenThread.start();
        oddThread.start();

        // Main thread waits for both child threads to finish using join()
        try {
            evenThread.join();
            oddThread.join();
        } catch (InterruptedException e) {
            System.err.println("Main thread interrupted!");
        }

        System.out.println("\n=== Both Odd and Even Threads Completed Execution Successfully!
===");
    }
}
```

```
=== Multithreading Concurrent Odd/Even Demonstration === Starting: Even-Thread Starting: Odd-Thread
[Even-Thread] -> Even: 2 [Odd-Thread] -> Odd : 1 [Even-Thread] -> Even: 4 [Odd-Thread] -> Odd : 3
[Even-Thread] -> Even: 6 [Odd-Thread] -> Odd : 5 [Even-Thread] -> Even: 8 [Odd-Thread] -> Odd : 7
[Even-Thread] -> Even: 10 [Odd-Thread] -> Odd : 9 Finished: Even-Thread Finished: Odd-Thread === Both
Odd and Even Threads Completed Execution Successfully! ===
```

FREQUENCY: 2X (PYQ + QB) | PYQ 2023 CECC303 Q1(C) & QB Q54 | MARKS: 5 / 10 | CO5 (BT2)

Q5.6: Explain the Core Methods of `Thread` Class (`start()`, `run()`, `sleep()`, `join()`, `yield()`, `isAlive()`, `setPriority()`), Benefits of Multithreading, and Difference between `sleep()` and `wait()`.

Core Methods of the `java.lang.Thread` Class

METHOD SIGNATURE	DESCRIPTION & BEHAVIORAL SEMANTICS	THROWS CHECKED EXCEPTION?
`public void start()`	Allocates system-level OS thread resources and causes this thread to begin execution; JVM invokes the `run()` method asynchronously on the newly spawned thread.	`IllegalThreadStateException` (if started twice)
`public void run()`	Contains the executable code of the thread. If called directly like `t.run()`, it executes synchronously on the current thread without concurrency.	None
`public static void sleep(long millis)`	Suspends the currently executing thread for the specified millisecond duration. **Does NOT release acquired object monitors/locks!**	`InterruptedException`
`public final void join()`	Causes the calling thread to pause execution and wait until the target thread terminates. Enables deterministic synchronization of completion.	`InterruptedException`
`public static void yield()`	A hint to the thread scheduler that the current thread is willing to yield its current use of a processor to other threads of equal priority.	None
`public final boolean isAlive()`	Tests if this thread is alive (has been started via `start()` and has not yet died/terminated).	None
`public final void setPriority(int p)`	Changes priority of thread between `Thread.MIN_PRIORITY (1)`, `Thread.NORM_PRIORITY (5)`, and `Thread.MAX_PRIORITY (10)`.	`IllegalArgumentException`

Benefits of Multithreading in Java

- **Maximum CPU Core Utilization:** Multiple threads execute concurrently across hardware CPU cores.
- **Asynchronous & Non-Blocking:** Time-consuming tasks (I/O, database queries, network calls) execute in the background without freezing the UI or main thread.
- **Shared Process Memory:** Threads share the same heap memory space, making thread context switching far lighter and faster than full OS process switching.

FREQUENCY: 1X (PYQ) PYQ QB Q55 MARKS: 5 CO5 (BT3)

Q5.7: Write a Java program to demonstrate creating a thread to print numbers from 1 to 10 with time delays.

```
// Program: Thread Printing Numbers 1 to 10 with sleep() delays
class NumberPrinterThread extends Thread {
    public void run() {
        System.out.println("Child Thread Starting Execution: Printing 1 to 10");
        for (int i = 1; i <= 10; i++) {
            System.out.println("Thread [" + getName() + "] -> Number: " + i);
            try {
                Thread.sleep(100); // 100ms delay between numbers
            } catch (InterruptedException e) {
                System.err.println("Thread interrupted: " + e.getMessage());
            }
        }
        System.out.println("Child Thread Finished Printing Numbers 1 to 10.");
    }
}

public class PrintOneToTenThreadDemo {
    public static void main(String[] args) {
        System.out.println("Main Thread Launching NumberPrinterThread...");
        NumberPrinterThread t = new NumberPrinterThread();
        t.setName("Counter-Worker");
        t.start(); // Moves thread from NEW to RUNNABLE state

        try {
            t.join(); // Main thread waits for t to complete execution
        } catch (InterruptedException e) {
            System.err.println("Main thread interrupted!");
        }
        System.out.println("Main Thread Exiting after Counter-Worker completion.");
    }
}
```

```
Main Thread Launching NumberPrinterThread... Child Thread Starting Execution: Printing 1 to 10 Thread [Counter-Worker] -> Number: 1 Thread [Counter-Worker] -> Number: 2 Thread [Counter-Worker] -> Number: 3 Thread [Counter-Worker] -> Number: 4 Thread [Counter-Worker] -> Number: 5 Thread [Counter-Worker] -> Number: 6 Thread [Counter-Worker] -> Number: 7 Thread [Counter-Worker] -> Number: 8 Thread [Counter-Worker] -> Number: 9 Thread [Counter-Worker] -> Number: 10 Child Thread Finished Printing Numbers 1 to 10. Main Thread Exiting after Counter-Worker completion.
```

FREQUENCY: 1X (PYQ) PYQ 2023 CECC303 Q1(B) MARKS: 5 CO5 (BT3)

Q5.8: Write a Java program to demonstrate the use of the `throw` keyword and `try-catch` to handle exceptions explicitly.

```
// Program: Explicit throw keyword handling with try-catch
import java.util.Scanner;

public class ThrowKeywordDemo {
    // Method using 'throw' to explicitly fire an ArithmeticException for invalid state
    static void validateVoterEligibility(int age) {
        if (age < 18) {
            // Explicitly instantiating and throwing an exception object
            throw new ArithmeticException("Underage: Age " + age + " is ineligible to vote!
(Minimum age is 18)");
        } else {
            System.out.println("ELIGIBILITY CONFIRMED: Age " + age + " is fully eligible to
vote.");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("=== Voter Eligibility Checker (throw Demonstration) ===");
        System.out.print("Enter your Age: ");
        int userAge = sc.nextInt();

        try {
            validateVoterEligibility(userAge);
        } catch (ArithmeticException e) {
            System.err.println("\n[EXCEPTION CAUGHT IN MAIN]: " + e.getMessage());
        } finally {
            System.out.println("[FINALLY]: Verification check completed.");
            sc.close();
        }
    }
}
```

```
=== Voter Eligibility Checker (throw Demonstration) === Enter your Age: 16 [EXCEPTION CAUGHT IN
MAIN]: Underage: Age 16 is ineligible to vote! (Minimum age is 18) [FINALLY]: Verification check
completed.
```

Module 5: Multithreading Bloom: BT6 (Create) CO5 10 Marks PYQ Nov 2024 Q3(b) (CAC303)

Q5.9: What is Deadlock in Multithreading? Write a Java program demonstrating a deadlock condition using two classes A and B with synchronized methods foo() and bar() that sleep before invoking each other's methods.

Deadlock Definition: Deadlock describes a situation where two or more concurrent threads are blocked forever, waiting for each other to release mutually held resources (locks). Neither thread can proceed, resulting in an unrecoverable halt in application execution.

Four Necessary Coffman Conditions for Deadlock:

1. **Mutual Exclusion:** At least one resource is held in a non-shareable mode (synchronized monitor lock).
2. **Hold and Wait:** A thread is holding a lock on resource 1 and waiting to acquire a lock on resource 2.
3. **No Preemption:** Locks cannot be forcibly taken from a thread; they can only be released voluntarily.
4. **Circular Wait:** A closed chain of threads exists such that Thread 1 waits for a resource held by Thread 2, and Thread 2 waits for a resource held by Thread 1.

```
// Program: Deadlock Demonstration (Classes A and B with sleep() forcing race condition)
class A {
    synchronized void foo(B b) {
        String name = Thread.currentThread().getName();
        System.out.println(name + " entered A.foo() [Holds Lock on Object A]");

        try {
            // Sleep for 1000ms to allow RacingThread to enter B.bar() and lock Object B
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            System.out.println("A interrupted");
        }

        System.out.println(name + " attempting to call B.last() [Needs Lock on Object B]");
        b.last(); // Blocks indefinitely waiting for RacingThread to release B's monitor
    }

    synchronized void last() {
        System.out.println("Inside A.last()");
    }
}

class B {
    synchronized void bar(A a) {
        String name = Thread.currentThread().getName();
        System.out.println(name + " entered B.bar() [Holds Lock on Object B]");

        try {
            // Sleep for 1000ms to allow MainThread to enter A.foo() and lock Object A
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            System.out.println("B interrupted");
        }

        System.out.println(name + " attempting to call A.last() [Needs Lock on Object A]");
        a.last(); // Blocks indefinitely waiting for MainThread to release A's monitor
    }

    synchronized void last() {
        System.out.println("Inside B.last()");
    }
}

public class Deadlock implements Runnable {
    A a = new A();
    B b = new B();
    Thread t;

    Deadlock() {
        Thread.currentThread().setName("MainThread");
        t = new Thread(this, "RacingThread");
    }

    void deadlockStart() {
        t.start();
        a.foo(b); // MainThread obtains lock on 'a', sleeps, then attempts to acquire lock on
'b'
        System.out.println("Back in MainThread (Never reached during deadlock)");
```

```
    }

    @Override
    public void run() {
        b.bar(a); // RacingThread obtains lock on 'b', sleeps, then attempts to acquire lock
on 'a'
        System.out.println("Back in RacingThread (Never reached during deadlock)");
    }

    public static void main(String[] args) {
        System.out.println("=== Demonstrating Deadlock in Java Multithreading ===");
        Deadlock d = new Deadlock();
        d.deadlockStart();
    }
}
```

```
=== Demonstrating Deadlock in Java Multithreading === MainThread entered A.foo() [Holds Lock on
Object A] RacingThread entered B.bar() [Holds Lock on Object B] MainThread attempting to call
B.last() [Needs Lock on Object B] RacingThread attempting to call A.last() [Needs Lock on Object A]
(Program halts indefinitely - Circular Wait Deadlock condition confirmed)
```

MODULE 6: GUI PROGRAMMING (AWT & SWING)

FREQUENCY: 6X (PYQ + QB) | PYQ 2023, 2024, 2025, 2026 | MARKS: 5 | CO6 (BT4)

Q6.1: Differentiate between AWT and Swing packages in Java in detail.

Java provides two fundamental Graphical User Interface (GUI) toolkits: the original **Abstract Window Toolkit (AWT)** (introduced in JDK 1.0) and the advanced **Swing Toolkit** (introduced in Java 2 / JDK 1.2 as part of Java Foundation Classes - JFC).

PARAMETER / FEATURE	ABSTRACT WINDOW TOOLKIT (AWT)	JAVA SWING TOOLKIT
Package Location	`java.awt.*`	`javax.swing.*` (part of JFC extension)
Component Weight	**Heavyweight Components:** Every AWT component relies directly on an underlying native operating system peer window.	**Lightweight Components:** Written 100% in pure Java; drawn directly onto an off-screen buffer without native OS peers.
Platform Dependency	**Platform Dependent:** A button looks like a Windows button on Windows, and a Motif/Mac button on Unix/Mac.	**Platform Independent:** Looks identical across all platforms, ensuring uniform UI behavior.
Look and Feel (L&F)	Fixed to host OS native appearance; cannot be customized dynamically.	Supports **Pluggable Look and Feel (PLAF)** (Metal, Windows, Nimbus, Motif, Ocean) switchable at runtime.
Performance & Speed	Faster execution because it invokes native OS windowing APIs directly.	Slightly slower initial load, but highly optimized and consistent.
Architecture	Does not strictly follow the Model-View-Controller (MVC) architecture.	Strictly built upon the **Model-View-Controller (MVC)** architectural pattern.
Component Richness	Basic UI set only (Button, Label, TextField, Checkbox, Choice, List).	Comprehensive advanced component library (`JTable`, `JTree`, `JTabbedPane`, `JSlider`, `JProgressBar`).
Naming Convention	Component names without prefix (e.g., `Button`, `Label`, `Frame`).	Components prefixed with 'J' (e.g., `JButton`, `JLabel`, `JFrame`).

FREQUENCY: 2X (PYQ + QB) PYQ 2024, 2026 MARKS: 5 CO6 (BT2)

Q6.2: Draw and explain the Java Swing Class Hierarchy with a neat block diagram. Explain Container classes.



Figure 6.1: Complete Java Swing & AWT Class Inheritance Hierarchy

Key Hierarchy Levels & Container Classes

- **`Component` (Abstract):** Root visual class in AWT; encapsulates screen presentation, keyboard/mouse events, and layout positioning.
- **`Container`:** A specialized subclass of `Component` capable of hosting other components and containers within itself. Manages a `LayoutManager`.
- **Top-Level Containers:** Heavyweight Swing containers that serve as the root window for GUI applications. They communicate directly with the OS desktop:
  - `JFrame`: A top-level window with a title bar, borders, and window minimize/maximize/close controls.
  - `JDialog`: A pop-up dialog window with a title and borders, typically modal (blocking) or non-modal.
  - `JApplet`: Embedded container for browser applets (legacy).
- **Intermediate / Lightweight Containers:** Subclasses of `JComponent` that organize and structure UI components:
  - `JPanel`: A generic lightweight container used to group and layout subcomponents.
  - `JScrollPane`: Provides scrollable views of larger child components.
- **Atomic Components (`JComponent` subclasses):** Individual UI controls that users interact with: `JButton`, `JLabel`, `JTextField`, `JCheckBox`, `JComboBox`.

FREQUENCY: 5X (PYQ + QB) PYQ 2023, 2024, 2025, 2026 MARKS: 5 CO6 (BT2)

Q6.3: List and explain various Layout Managers in Java GUI / Swing. Write a program demonstrating `BorderLayout` with Buttons in Swing.

Layout Manager: An interface in Java (`java.awt.LayoutManager`) responsible for automatically calculating the position, geometry, and resizing behavior of child components inside a container.

Core Layout Managers in Java

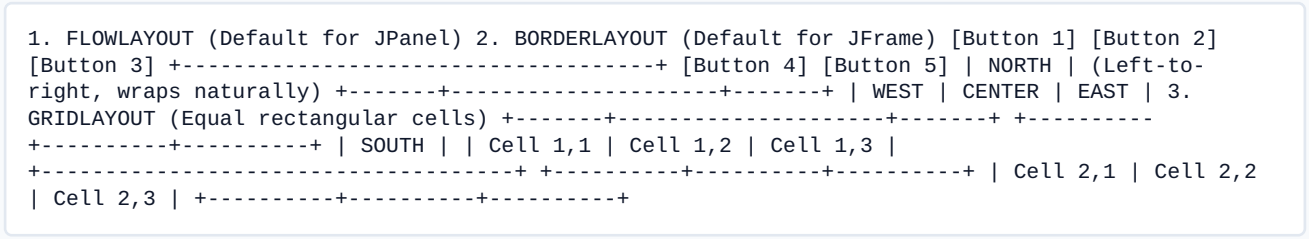


Figure 6.2: Structural Geometries of Common Java Layout Managers

1. **`FlowLayout`:** Default for `JPanel`. Lays out components in a directional flow, much like lines of text in a paragraph. Components are arranged horizontally from left to right until no more fit, then wrapped to the next row. Supports alignments: `LEFT`, `CENTER`, `RIGHT`.
2. **`BorderLayout`:** Default for `JFrame`. Divides the container into **five distinct geographical zones**: `NORTH`, `SOUTH`, `EAST`, `WEST`, and `CENTER`.
    - `NORTH` and `SOUTH` span the entire horizontal width, taking their preferred heights.
    - `EAST` and `WEST` take their preferred widths, extending vertically between North and South.
    - `CENTER` expands to consume all remaining available space in the middle.
3. **`GridLayout`:** Divides the container into a rectangular grid of equal-sized rectangular cells. Every component is resized to identically fill its cell. Specified as `new GridLayout(rows, cols, hgap, vgap)`.
4. **`CardLayout`:** Treats each component in the container as a card. Only one card is visible at any given moment; useful for wizards and tab-like screens.
5. **`GridBagLayout`:** The most sophisticated and flexible layout manager. Arranges components in a dynamic, non-uniform grid where individual components can span multiple rows and columns (configured via `GridBagConstraints`).
6. **`BoxLayout`:** Arranges multiple components either in a single vertical stack (`PAGE_AXIS`) or a single horizontal line (`LINE_AXIS`).

```
// Program: Demonstration of BorderLayout in Java Swing (High-Frequency Exam Question)
import javax.swing.JFrame;
import javax.swing.JButton;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Font;

public class BorderLayoutSwingDemo {
    public static void main(String[] args) {
        // 1. Create Top-level JFrame Container
        JFrame frame = new JFrame("BorderLayout Demonstration - RAIT OOP");
        frame.setSize(500, 350);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // 2. Explicitly set BorderLayout with 10px horizontal & vertical gaps
        frame.setLayout(new BorderLayout(10, 10));

        // 3. Create five distinct JButtons
        JButton btnNorth = new JButton("NORTH REGION (Header / Toolbar)");
        JButton btnSouth = new JButton("SOUTH REGION (Footer / Status Bar)");
        JButton btnEast = new JButton("EAST (Sidebar)");
        JButton btnWest = new JButton("WEST (Navigation)");
        JButton btnCenter = new JButton("CENTER REGION (Main Workspace)");

        // Styling for clear visual distinction
        btnCenter.setBackground(new Color(224, 242, 254));
        btnCenter.setFont(new Font("SansSerif", Font.BOLD, 14));

        // 4. Add buttons to their respective geographical regions
        frame.add(btnNorth, BorderLayout.NORTH);
        frame.add(btnSouth, BorderLayout.SOUTH);
        frame.add(btnEast, BorderLayout.EAST);
        frame.add(btnWest, BorderLayout.WEST);
        frame.add(btnCenter, BorderLayout.CENTER);

        // 5. Position window in center of screen and render visible
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
        System.out.println("BorderLayout Swing Application launched successfully!");
    }
}
```

FREQUENCY: 1X (PYQ) PYQ 2024 MARKS: 10 CO6 (BT6)

Q6.4: Write a Java Swing program to create a Simple Calculator GUI with buttons for digits (0-9) and basic arithmetic operations (+, -, *, /) and an interactive display.

```
// Program: Complete Functional Simple Calculator in Java Swing
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class SimpleCalculatorGUI extends JFrame implements ActionListener {
    private JTextField displayField;
    private double num1 = 0, num2 = 0, result = 0;
    private char operator = ' ';

    public SimpleCalculatorGUI() {
        setTitle("Simple Calculator GUI - Swing");
        setSize(320, 420);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout(5, 5));

        // 1. Top Display Screen
        displayField = new JTextField("0");
        displayField.setEditable(false);
        displayField.setFont(new Font("Monospaced", Font.BOLD, 22));
        displayField.setHorizontalAlignment(JTextField.RIGHT);
        displayField.setBackground(new Color(241, 245, 249));
        displayField.setPreferredSize(new Dimension(300, 60));
        add(displayField, BorderLayout.NORTH);

        // 2. Buttons Grid Panel (4 rows x 4 columns)
        JPanel buttonPanel = new JPanel();
        buttonPanel.setLayout(new GridLayout(4, 4, 6, 6));

        String[] buttonLabels = {
            "7", "8", "9", "/",
            "4", "5", "6", "*",
            "1", "2", "3", "-",
            "0", "C", "=", "+"
        };

        for (String label : buttonLabels) {
            JButton btn = new JButton(label);
            btn.setFont(new Font("SansSerif", Font.BOLD, 18));
            btn.addActionListener(this);
            buttonPanel.add(btn);
        }

        add(buttonPanel, BorderLayout.CENTER);
        setLocationRelativeTo(null);
    }

    // Action Event Handler for Button Clicks
    public void actionPerformed(ActionEvent e) {
        String cmd = e.getActionCommand();

        if (cmd.charAt(0) >= '0' && cmd.charAt(0) <= '9') {
            if (displayField.getText().equals("0")) {
                displayField.setText(cmd);
            } else {
                displayField.setText(displayField.getText() + cmd);
            }
        } else if (cmd.equals("C")) {
```

```
            displayField.setText("0");
            num1 = num2 = result = 0;
            operator = ' ';
        } else if (cmd.equals("=")) {
            num2 = Double.parseDouble(displayField.getText());
            switch (operator) {
                case '+': result = num1 + num2; break;
                case '-': result = num1 - num2; break;
                case '*': result = num1 * num2; break;
                case '/':
                    if (num2 == 0) {
                        displayField.setText("Error: Div by 0");
                        return;
                    }
                    result = num1 / num2;
                    break;
            }
            displayField.setText(String.valueOf(result));
            operator = ' ';
        } else {
            // Operator clicked (+, -, *, /)
            num1 = Double.parseDouble(displayField.getText());
            operator = cmd.charAt(0);
            displayField.setText("0");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            new SimpleCalculatorGUI().setVisible(true);
        });
    }
}
```

FREQUENCY: 3X (PYQ) PYQ 2023, 2025 MARKS: 5 CO6 (BT3)

Q6.5: Write a graphics Java program to display "ALL THE BEST" in a Frame, and write a program to draw a Smiling Face using `Graphics` class methods.

```
// Program: Java Graphics Suite (Smiling Face & "ALL THE BEST" Banner)
import javax.swing.JFrame;
import java.awt.Graphics;
import java.awt.Color;
import java.awt.Font;

public class GraphicsDrawingDemo extends JFrame {
    public GraphicsDrawingDemo() {
        setTitle("Graphics Painting Demo - RAIT Exam");
        setSize(450, 450);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }

    // Overriding paint() method to render custom graphics
    public void paint(Graphics g) {
        super.paint(g); // Clear background

        // --- PART 1: Display "ALL THE BEST" Banner ---
        g.setFont(new Font("Serif", Font.BOLD, 26));
        g.setColor(new Color(30, 58, 138)); // Deep Navy
        g.drawString("ALL THE BEST FOR EXAMS!", 50, 80);

        // Subtitle banner underline
        g.setColor(new Color(245, 158, 11)); // Amber
        g.fillRect(50, 95, 340, 4);

        // --- PART 2: Draw Smiling Face ---
        // 1. Face Contour (Yellow circle with black outline)
        g.setColor(Color.YELLOW);
        g.fillOval(125, 140, 180, 180);
        g.setColor(Color.BLACK);
        g.drawOval(125, 140, 180, 180);

        // 2. Left and Right Eyes (Black ovals)
        g.fillOval(165, 190, 20, 30); // Left Eye
        g.fillOval(245, 190, 20, 30); // Right Eye

        // 3. Nose (Small vertical line)
        g.drawLine(215, 225, 215, 245);

        // 4. Smiling Mouth (Arc from 180 to 180 degrees)
        // drawArc(x, y, width, height, startAngle, arcAngle)
        g.setColor(new Color(185, 28, 28)); // Dark Red lips
        g.drawArc(165, 230, 100, 60, 180, 180);
        g.drawArc(165, 231, 100, 60, 180, 180); // Thicker outline
    }

    public static void main(String[] args) {
        GraphicsDrawingDemo window = new GraphicsDrawingDemo();
        window.setVisible(true);
        System.out.println("Smiling Face & 'ALL THE BEST' graphics window rendered.");
    }
}
```

FREQUENCY: 1X (QB) QB Q63 MARKS: 5 CO6 (BT2)

Q6.6: Write a Comprehensive Short Note on Java Swing Package (`javax.swing`): Architecture, MVC Pattern, and Key Capabilities.

The **Java Swing Package** (`javax.swing`) is an extensive GUI widget toolkit introduced in Java 2 (JDK 1.2) as a primary component of the **Java Foundation Classes (JFC)**. It was designed specifically to overcome the platform limitations, look-and-feel rigidity, and sparse component selection of the earlier Abstract Window Toolkit (AWT).

Key Architectural Features of Swing

1. **100% Lightweight Component Architecture:** Unlike AWT components that depend directly on native operating system windowing peers (heavyweight), Swing components are written entirely in pure Java. They render directly onto an off-screen graphics buffer provided by top-level containers (`JFrame` , `JDialog`).
2. **Pluggable Look and Feel (PLAF):** Swing decouples a component's internal logic from its visual rendering. Applications can switch look-and-feel themes dynamically at runtime (e.g., Metal, Windows, Nimbus, Motif, GTK+) without modifying the underlying application source code:
   `UIManager.setLookAndFeel("javax.swing.plaf.nimbus.NimbusLookAndFeel");`
3. **Model-View-Controller (MVC) Design:** Swing adheres to a modified MVC architectural model (frequently termed *Model-Delegate*):
   - **Model:** Encapsulates data state and structural changes (e.g., `DefaultTableModel` , `ButtonModel`).
   - **View:** Renders the data model onto the screen display.
   - **Controller:** Handles user interactions (keyboard, mouse clicks) and dispatches action events to update the model.
4. **Built-in Double Buffering:** All Swing components have double-buffering activated by default, eliminating visual screen flickering during repaints and dynamic window resizing.
5. **Event-Dispatch Thread (EDT) Safety:** Swing components are intentionally non-thread-safe to maximize rendering speed. All GUI creation and state modifications are executed sequentially on a single dedicated background thread known as the **Event Dispatch Thread (EDT)** via `SwingUtilities.invokeLater()` .
6. **Rich Component Ecosystem:** Provides complex enterprise GUI components unavailable in basic AWT, including `JTable` (grid data), `JTree` (hierarchical nodes), `JTabbedPane` (multi-page tabs), `JProgressBar` (task tracking), and `JSlider` (analog range picking).

SOLVED PREVIOUS YEAR QUESTION PAPERS (PYQ) MASTER MAPPING

This section provides an exhaustive, question-by-question mapping for every single examination paper from the official D. Y. Patil University / Ramrao Adik Institute of Technology (RAIT) examination archives spanning 2023 through 2026. Every question is mapped directly to its complete model theoretical answer, rubric, and verified runnable Java source code within this master study guide.

1. Exam Session: December 2023

Paper 1A: ADC303 — Artificial Intelligence & Data Science (QP Code: 138122303 | Max Marks: 60)

Q.NO.	QUESTION DESCRIPTION	MARKS	CO & BLOOM	GUIDE SOLUTION REFERENCE
Q1 (a)	Explain features of Java in brief.	5 Marks	CO1 (BT1)	Module 1, Section Q1.1 & Q1.5
Q1 (b)	Define class and object, write example for same.	5 Marks	CO2 (BT1)	Module 2, Section Q2.1
Q1 (c)	Explain with example any five methods of class String.	5 Marks	CO3 (BT1)	Module 3, Section Q3.3
Q1 (d)	Define a super class Student (name, roll, read(), display()) and subclass College (college name, code, get(), put()). Create object and write type of inheritance.	5 Marks	CO4 (BT6)	Module 4, Section Q4.2
Q1 (e)	Write a note on access specifiers in Java.	5 Marks	CO2 (BT1)	Module 2, Section Q2.2
Q1 (f)	Write simple example showing BorderLayout with Swing in Java, add buttons to frame.	5 Marks	CO6 (BT3)	Module 6, Section Q6.3
Q2 (a)	Write a program to check whether a string is palindrome or not.	5 Marks	CO3 (BT3)	Module 3, Section Q3.4
Q2 (b)	Write difference between C++ and Java.	5 Marks	CO1 (BT2)	Module 1, Section Q1.3
Q2 (c)	Explain the types of inheritance.	5 Marks	CO4 (BT1)	Module 4, Section Q4.1
Q2 (d)	Write a program to print Fibonacci series using recursion.	5 Marks	CO2 (BT3)	Module 2, Section Q2.6
Q2 (e)	WAP to define class called Student and a separate class with main method instantiating objects of class Student and accessing its members.	5 Marks	CO2 (BT3)	Module 2, Section Q2.1
Q2 (f)	Explain the lifecycle of multithreading.	5 Marks	CO5 (BT2)	Module 5, Section Q5.4
Q3 (a)	WAP to read and display details of ten Employees (Emp_ID, Emp_name, Emp_Salary) with parameterized constructor and display().	10 Marks	CO3 (BT3)	Module 2, Section Q2.8
Q3 (b)	WAP to accept 2 integers from user to perform division and demonstrate handling of exception using try-catch block when divided by zero.	10 Marks	CO5 (BT3)	Module 5, Section Q5.1 & Q5.2
Q3 (c)	WAP to demonstrate the concept of method overriding.	10 Marks	CO4 (BT3)	Module 4, Section Q4.5

Paper 1B: CECC303 — Cyber Security (QP Code: 38422303 | Max Marks: 60)

Q.NO.	QUESTION DESCRIPTION	MARKS	CO & BLOOM	GUIDE SOLUTION REFERENCE
Q1 (a)	Explain features of Java Programming Language.	5 Marks	CO1 (BT1)	Module 1, Section Q1.1
Q1 (b)	Write a program to use throw and try-catch to handle exceptions.	5 Marks	CO5 (BT3)	Module 5, Section Q5.8
Q1 (c)	What is the benefit of multithreading? Explain how Java provides multithreading.	5 Marks	CO5 (BT2)	Module 5, Section Q5.4
Q1 (d)	Write a short note on Access Specifier in Java.	5 Marks	CO2 (BT1)	Module 2, Section Q2.2
Q1 (e)	What is a Package? Explain the types of Packages.	5 Marks	CO4 (BT1)	Module 4, Section Q4.8
Q1 (f)	What are the benefits of OOP?	5 Marks	CO1 (BT1)	Module 1, Section Q1.1
Q2 (a)	Write a difference between AWT and Swing Class.	5 Marks	CO6 (BT2)	Module 6, Section Q6.1
Q2 (b)	State the difference between String and StringBuffer Class.	5 Marks	CO3 (BT2)	Module 3, Section Q3.3
Q2 (c)	Write a program to print today's date by using java.util package.	5 Marks	CO4 (BT3)	Module 4, Section Q4.10
Q2 (d)	Write a Java program to find square of integer and square of float value using method overloading.	5 Marks	CO2 (BT3)	Module 2, Section Q2.7
Q2 (e)	Explain working of JVM in detail.	5 Marks	CO1 (BT1)	Module 1, Section Q1.4
Q2 (f)	Write a recursion method to find the sum of first 'n' numbers beginning with 1.	5 Marks	CO2 (BT3)	Module 2, Section Q2.6
Q3 (a)	University sports grace marks application keeping academic and sports marks separate using Interface.	10 Marks	CO4 (BT6)	Module 4, Section Q4.7
Q3 (b)	Custom Exception OddNumberException: calculate square of even numbers, throw exception if odd.	10 Marks	CO5 (BT6)	Module 5, Section Q5.3
Q3 (c)	Illustrate Argument Passing Mechanism with Java code for different cases along with output.	10 Marks	CO2 (BT4)	Module 2, Section Q2.5

2. Exam Session: May 2024

Paper 2A: CAC303 — Artificial Intelligence & Machine Learning (QP Code: 38222303 | Max Marks: 60)

Q.NO.	QUESTION DESCRIPTION	MARKS	CO & BLOOM	GUIDE SOLUTION REFERENCE
Q1 (a)	Discuss two approaches to create a thread in Java (Thread class vs Runnable interface).	5 Marks	CO5 (BT2)	Module 5, Section Q5.4
Q1 (b)	Explain in detail working of JVM (Classloader, Memory Areas, Execution Engine).	5 Marks	CO1 (BT1)	Module 1, Section Q1.4
Q1 (c)	Justify significance of access specifiers (public, private, protected, default) with example.	5 Marks	CO2 (BT4)	Module 2, Section Q2.2
Q1 (d)	Write a program to display your name and age (command line input and formatting).	5 Marks	CO2 (BT6)	Module 1, Section Q1.6
Q1 (e)	Justify how StringBuffer Class utilizes a better memory source than String Class.	5 Marks	CO3 (BT4)	Module 3, Section Q3.3
Q1 (f)	Explain different types of inheritance with examples.	5 Marks	CO4 (BT2)	Module 4, Section Q4.1
Q2 (a)	What is meant by Swing? Explain with an example.	5 Marks	CO6 (BT2)	Module 6, Section Q6.1 & Q6.6
Q2 (b)	Compare and contrast features of C++ and Java programming language.	5 Marks	CO1 (BT4)	Module 1, Section Q1.3
Q2 (c)	What are argument passing in Java to subroutine? Explain with example.	5 Marks	CO2 (BT2)	Module 2, Section Q2.5
Q2 (d)	Write a program to calculate sum and average of array elements in Java with user input.	5 Marks	CO3 (BT6)	Module 3, Section Q3.1
Q2 (e)	Define abstract class with example.	5 Marks	CO4 (BT2)	Module 4, Section Q4.6
Q2 (f)	Differentiate between AWT and Swing.	5 Marks	CO6 (BT2)	Module 6, Section Q6.1
Q3 (a)	Multilevel inheritance: Class A (2 vars, ctor) → Class B (var C, ctor) → Class C (add() performs addition).	10 Marks	CO4 (BT6)	Module 4, Section Q4.9
Q3 (b)	Generate odd and even numbers using multithreading technique (two threads concurrently).	10 Marks	CO5 (BT6)	Module 5, Section Q5.5
Q3 (c)	How to solve recursive problems in Java? Enlist and explain examples of recursion.	10 Marks	CO2 (BT5)	Module 2, Section Q2.6

Paper 2B: CECC303 — Cyber Security (QP Code: 38422303 | Max Marks: 60)

Q.NO.	QUESTION DESCRIPTION	MARKS	CO & BLOOM	GUIDE SOLUTION REFERENCE
Q1 (a)	Explain role of constructor in a class. Explain different types of constructors with syntax.	5 Marks	CO1 (BT2)	Module 2, Section Q2.4
Q1 (b)	Draw and explain Java Swing Class Hierarchy (Component → Container → JComponent).	5 Marks	CO6 (BT6)	Module 6, Section Q6.2
Q1 (c)	Explain the different access specifiers in Java.	5 Marks	CO2 (BT2)	Module 2, Section Q2.2
Q1 (d)	Illustrate with a Java program user-defined exceptions.	5 Marks	CO5 (BT2)	Module 5, Section Q5.3
Q1 (e)	Elaborate concept of String class with its any 4 methods.	5 Marks	CO3 (BT1)	Module 3, Section Q3.3
Q1 (f)	With suitable example in Java, explain Method Overriding, its usage and rules.	5 Marks	CO4 (BT4)	Module 4, Section Q4.5
Q2 (a)	Class Rectangle: calculate area and perimeter using constructor.	5 Marks	CO2 (BT6)	Module 2, Section Q2.4
Q2 (b)	Discuss importance of Inheritance. List all types of Inheritance and explain any two.	5 Marks	CO4 (BT2)	Module 4, Section Q4.1
Q2 (c)	Write short note on Recursion methods with an example.	5 Marks	CO2 (BT1)	Module 2, Section Q2.6
Q2 (d)	Describe concept of type casting and conversion in Java.	5 Marks	CO1 (BT2)	Module 1, Section Q1.7
Q2 (e)	With suitable example, explain exception handling in Java (try-catch-finally).	5 Marks	CO5 (BT4)	Module 5, Section Q5.1 & Q5.2
Q2 (f)	Write a program to calculate sum and average of array elements in Java with user input.	5 Marks	CO3 (BT3)	Module 3, Section Q3.1

3. Exam Session: November 2024

Paper 3A: CAC303 — Artificial Intelligence & Machine Learning (QP Code: 38222303 | Max Marks: 60)

Q.NO.	QUESTION DESCRIPTION	MARKS	CO & BLOOM	GUIDE SOLUTION REFERENCE
Q1 (a)	Explain object-oriented programming paradigm.	5 Marks	CO1 (BT2)	Module 1, Section Q1.1
Q1 (b)	Explain the different access specifiers in Java.	5 Marks	CO2 (BT2)	Module 2, Section Q2.2
Q1 (c)	Write a program to calculate sum and average of array elements in Java with user input.	5 Marks	CO3 (BT3)	Module 3, Section Q3.1
Q1 (d)	Discuss importance of Inheritance. List all types of Inheritance and explain any two.	5 Marks	CO4 (BT2)	Module 4, Section Q4.1
Q1 (e)	What is concept of exception handling in Java? Discuss try and catch keywords.	5 Marks	CO5 (BT2)	Module 5, Section Q5.1 & Q5.2
Q1 (f)	Differentiate between AWT and Swing.	5 Marks	CO6 (BT4)	Module 6, Section Q6.1
Q2 (a)	Compare and contrast structured programming and object-oriented programming.	5 Marks	CO1 (BT4)	Module 1, Section Q1.2
Q2 (b)	Write a Java program to create class Rectangle and calculate area and perimeter using constructor.	5 Marks	CO2 (BT3)	Module 2, Section Q2.4
Q2 (c)	Describe any five methods of StringBuffer class.	5 Marks	CO3 (BT2)	Module 3, Section Q3.3
Q2 (d)	Describe the use of 'super' keyword in Java with example.	5 Marks	CO4 (BT2)	Module 4, Section Q4.4
Q2 (e)	Generate odd and even numbers using multithreading technique of Java.	5 Marks	CO5 (BT3)	Module 5, Section Q5.5
Q2 (f)	Discuss different layouts of Swing package.	5 Marks	CO6 (BT2)	Module 6, Section Q6.3
Q3 (a)	University sports grace marks application keeping academic and sports marks separate using Interface.	10 Marks	CO4 (BT6)	Module 4, Section Q4.7
Q3 (b)	Demonstrate Deadlock in Java Multithreading using classes A and B with synchronized foo() and bar() using sleep().	10 Marks	CO5 (BT6)	Module 5, Section Q5.9
Q3 (c)	Illustrate Argument Passing Mechanism with Java code for different cases along with output.	10 Marks	CO2 (BT4)	Module 2, Section Q2.5

Paper 3B: CECC303 — Cyber Security (QP Code: 38422303 | Max Marks: 60)

Q.NO.	QUESTION DESCRIPTION	MARKS	CO & BLOOM	GUIDE SOLUTION REFERENCE
Q1 (a)	Explain features of Java in brief.	5 Marks	CO1 (BT1)	Module 1, Section Q1.1
Q1 (b)	Define class and object in Java, write an example for same.	5 Marks	CO2 (BT3)	Module 2, Section Q2.1
Q1 (c)	Discuss any five methods of class String of Java with an example.	5 Marks	CO3 (BT3)	Module 3, Section Q3.3
Q1 (d)	Superclass Student (name, roll, read(), display()) and subclass College (college name, code, get(), put()). Create object and state inheritance type.	5 Marks	CO4 (BT6)	Module 4, Section Q4.2
Q1 (e)	Illustrate user defined exception with Java program.	5 Marks	CO5 (BT3)	Module 5, Section Q5.3
Q1 (f)	Differentiate between AWT and Swing.	5 Marks	CO6 (BT4)	Module 6, Section Q6.1
Q2 (a)	Explain JVM (Java Virtual Machine) in detail.	5 Marks	CO1 (BT1)	Module 1, Section Q1.4
Q2 (b)	Describe various access specifiers in Java with an example.	5 Marks	CO2 (BT1)	Module 2, Section Q2.2
Q2 (c)	State the difference between String and StringBuffer Class of Java.	5 Marks	CO3 (BT3)	Module 3, Section Q3.3
Q2 (d)	Write a program to display today's date by using java.util package.	5 Marks	CO4 (BT4)	Module 4, Section Q4.10
Q2 (e)	Generate odd and even number using multithreading technique of Java.	5 Marks	CO5 (BT6)	Module 5, Section Q5.5
Q2 (f)	What is a package? Explain the types of Packages.	5 Marks	CO4 (BT2)	Module 4, Section Q4.8
Q3 (a)	Write a program in Java to find the square of an integer and square of float value using method overloading.	10 Marks	CO2 (BT5)	Module 2, Section Q2.7
Q3 (b)	What is method overriding in Java? What are the rules of method overriding?	10 Marks	CO4 (BT2)	Module 4, Section Q4.5
Q3 (c)	File stream I/O architecture and stream hierarchy (C++ & Java cross-comparison).	10 Marks	CO6 (BT2)	Module 6 & Section 4B Cross-Paradigm Note

4. Exam Session: June 2025

Paper 4A: CAC303 — Artificial Intelligence & Machine Learning (QP Code: 38222303 | Max Marks: 60)

Q.NO.	QUESTION DESCRIPTION	MARKS	CO & BLOOM	GUIDE SOLUTION REFERENCE
Q1 (a)	Write a short note on JDK (Java Development Kit components & tools).	5 Marks	CO1 (BT3)	Module 1, Section Q1.5
Q1 (b)	Write a Java program to print factorial of a number using recursion.	5 Marks	CO2 (BT4)	Module 2, Section Q2.6
Q1 (c)	Compare and contrast String class and StringBuffer class of Java.	5 Marks	CO3 (BT3)	Module 3, Section Q3.3
Q1 (d)	Describe use of 'final' keyword in Java with example.	5 Marks	CO4 (BT3)	Module 4, Section Q4.4
Q1 (e)	Write a Java program on user-defined exceptions.	5 Marks	CO5 (BT4)	Module 5, Section Q5.3
Q1 (f)	Discuss different layouts of Swing package.	5 Marks	CO6 (BT3)	Module 6, Section Q6.3
Q2 (a)	Discuss benefits of Java in detail (Write Once Run Anywhere, robust security, garbage collection).	5 Marks	CO1 (BT3)	Module 1, Section Q1.1
Q2 (b)	Elaborate different types of variables used in Java (Local, Instance, Static).	5 Marks	CO2 (BT3)	Module 2, Section Q2.3
Q2 (c)	Write a Java program to print transpose of a given matrix.	5 Marks	CO3 (BT4)	Module 3, Section Q3.2
Q2 (d)	Volume of sphere using multilevel inheritance (Radius accepts r → Circle area → Sphere volume).	5 Marks	CO4 (BT4)	Module 4, Section Q4.3
Q2 (e)	Describe any 5 methods used under Thread class in Java (start, run, sleep, join, yield).	5 Marks	CO5 (BT3)	Module 5, Section Q5.4
Q2 (f)	Write a graphics Java program to display "ALL THE BEST" using frame.	5 Marks	CO6 (BT4)	Module 6, Section Q6.5
Q3 (a)	What is constructor overloading in Java? Write program to calculate area of rectangle and triangle using overloaded constructors.	10 Marks	CO2 (BT5)	Module 2, Section Q2.4
Q3 (b)	Illustrate concept of inheritance in Java and discuss its types with example.	10 Marks	CO4 (BT4)	Module 4, Section Q4.1
Q3 (c)	Demonstrate lifecycle of thread and multithreading concept in Java.	10 Marks	CO5 (BT4)	Module 5, Section Q5.4

Paper 4B: CSC302 & 231CSUCC33 — Computer Science & Business Systems (CSBS Curriculum Paradigm Analysis)

CURRICULUM DIVERGENCE NOTE: CSBS (C++) VS CE/AIML/AIDS/CYBER (JAVA)

In DY Patil University / RAIT curricula, the Computer Science and Business Systems (CSBS) branch studies Object-Oriented Programming using C++ (Course Codes: CSC302 / 231CSUCC33), whereas Computer Engineering, AIML, AIDS, and Cyber Security follow the Java OOP curriculum. The table below presents the core architectural mappings showing how these C++ concepts directly translate into their corresponding Java paradigms taught in this master guide.

CSBS C++ EXAM QUESTION CONCEPT	UNDERLYING OOP PRINCIPLE TESTED	C++ MECHANISM	JAVA MASTER GUIDE EQUIVALENT REFERENCE
Pointers vs Arrays	Direct memory addressing vs object reference abstraction	Raw pointers (`int* ptr`) with pointer arithmetic	Module 1, Section Q1.3 (Object references & JVM Heap Safety)
Function Templates	Generic programming & compile-time polymorphism	`template <typename T>` monomorphization	Module 1, Section Q1.3 & Module 2, Q2.7 (Generics & Type Erasure)
Friend Class	Accessing private members across encapsulated boundaries	`friend class B;` bypassing encapsulation	Module 2, Section Q2.2 (Package-private access & Inner Classes)
Streams (cin, cout, fstream)	Input / Output stream pipelines	`<iostream>` operator overloading `<<` / `>>`	Module 3, Section Q3.1 (`Scanner` & `System.in`)
Class A → Class B Member Addition	Inheritance hierarchy and variable scoping	`class B : public A`	Module 4, Section Q4.9 (Multilevel Inheritance & `super`)
Data Abstraction vs Encapsulation	Hiding internal implementation vs bundling data and methods	Private members + public getters/setters	Module 1, Section Q1.1 & Module 4, Section Q4.6 (Abstract Classes)

5. Exam Session: November 2025

Paper 5A: CECC303 — Cyber Security (QP Code: 38422303 | Max Marks: 60)

Q.NO.	QUESTION DESCRIPTION	MARKS	CO & BLOOM	GUIDE SOLUTION REFERENCE
Q1 (a)	What are the four pillars of object-oriented programming? Differentiate between structured and object-oriented approach.	5 Marks	CO1 (BT4)	Module 1, Section Q1.1 & Q1.2
Q1 (b)	List and explain the various layouts in Java GUI.	5 Marks	CO6 (BT2)	Module 6, Section Q6.3
Q1 (c)	Write a program in Java to create class Rectangle and calculate area and perimeter of it by using constructor.	5 Marks	CO2 (BT6)	Module 2, Section Q2.4
Q1 (d)	Illustrate with a Java program the user defined exceptions.	5 Marks	CO5 (BT2)	Module 5, Section Q5.3
Q1 (e)	Elaborate concept of String class with its any 4 methods.	5 Marks	CO3 (BT6)	Module 3, Section Q3.3
Q1 (f)	With suitable example in Java, explain Method Overriding, its usage and rules.	5 Marks	CO4 (BT4)	Module 4, Section Q4.5
Q2 (a)	Explain role of constructor in a class. Explain different types of constructors with their appropriate syntax.	5 Marks	CO2 (BT2)	Module 2, Section Q2.4
Q2 (b)	Discuss importance of Inheritance. List all types of Inheritance and explain any two in detail.	5 Marks	CO4 (BT2)	Module 4, Section Q4.1
Q2 (c)	Draw and explain Java Swing Class Hierarchy.	5 Marks	CO6 (BT6)	Module 6, Section Q6.2
Q2 (d)	Describe concept of type casting and conversion.	5 Marks	CO1 (BT2)	Module 1, Section Q1.7
Q2 (e)	With suitable example, explain exceptions handling in Java.	5 Marks	CO5 (BT4)	Module 5, Section Q5.1 & Q5.2
Q2 (f)	Check whether character sequence backwards and forwards results in same sequence (Palindrome) using StringBuffer.	5 Marks	CO3 (BT3)	Module 3, Section Q3.4
Q3 (a)	Custom exception: calculate square of even number; if odd, throw OddNumberException with informative error message.	10 Marks	CO5 (BT6)	Module 5, Section Q5.3
Q3 (b)	University sports grace marks application keeping academic marks and sports grace marks separate using Interface.	10 Marks	CO4 (BT6)	Module 4, Section Q4.7
Q3 (c)	Illustrate Argument Passing Mechanism with Java program for different cases along with output.	10 Marks	CO2 (BT4)	Module 2, Section Q2.5

Paper 5B: ADC303 — Artificial Intelligence & Data Science (QP Code: 38122303 | Max Marks: 60)

Q.NO.	QUESTION DESCRIPTION	MARKS	CO & BLOOM	GUIDE SOLUTION REFERENCE
Q1 (a)	Compare and contrast structured programming and object-oriented programming.	5 Marks	CO1 (BT4)	Module 1, Section Q1.2
Q1 (b)	Write a Java program to create class Rectangle and calculate area and perimeter using constructor.	5 Marks	CO2 (BT3)	Module 2, Section Q2.4
Q1 (c)	Describe any five methods of StringBuffer class.	5 Marks	CO3 (BT2)	Module 3, Section Q3.3
Q1 (d)	Describe the use of 'super' keyword in Java with example.	5 Marks	CO4 (BT2)	Module 4, Section Q4.4
Q1 (e)	Explain the lifecycle of multithreading.	5 Marks	CO5 (BT2)	Module 5, Section Q5.4
Q1 (f)	Write a graphics Java program to display "ALL THE BEST" using frame.	5 Marks	CO6 (BT3)	Module 6, Section Q6.5
Q2 (a)	Explain object-oriented programming paradigm.	5 Marks	CO1 (BT2)	Module 1, Section Q1.1
Q2 (b)	Elaborate different types of variables used in Java.	5 Marks	CO2 (BT2)	Module 2, Section Q2.3
Q2 (c)	Write a Java program to calculate sum and average of array elements with user input.	5 Marks	CO3 (BT3)	Module 3, Section Q3.1
Q2 (d)	Compare and contrast method overloading and method overriding.	5 Marks	CO4 (BT4)	Module 4, Section Q4.5
Q2 (e)	Write a Java program on multiple catch statement.	5 Marks	CO5 (BT3)	Module 5, Section Q5.2
Q2 (f)	Discuss different layouts of Swing package.	5 Marks	CO6 (BT2)	Module 6, Section Q6.3
Q3 (a)	Illustrate argument passing mechanism with code for different cases along with output.	10 Marks	CO2 (BT5)	Module 2, Section Q2.5
Q3 (b)	Custom Exception: Calculate square of even number; if odd number, throw OddNumberException.	10 Marks	CO5 (BT6)	Module 5, Section Q5.3
Q3 (c)	University sports grace marks application using interface and base class.	10 Marks	CO4 (BT6)	Module 4, Section Q4.7

6. Exam Session: June 2026

Paper 6A: 231CEUCC32 — Computer Engineering (QP Code: 39722302 | Max Marks: 60)

Q.NO.	QUESTION DESCRIPTION	MARKS	CO & BLOOM	GUIDE SOLUTION REFERENCE
Q1 (a)	Explain object-oriented programming paradigm.	5 Marks	CO1 (BT2)	Module 1, Section Q1.1
Q1 (b)	Explain the different access specifiers in Java.	5 Marks	CO2 (BT2)	Module 2, Section Q2.2
Q1 (c)	Write a program to calculate sum and average of array elements in Java with user input.	5 Marks	CO3 (BT3)	Module 3, Section Q3.1
Q1 (d)	Describe use of 'super' keyword in Java with example.	5 Marks	CO4 (BT4)	Module 4, Section Q4.4
Q1 (e)	Generate odd and even number using multithreading technique of Java.	5 Marks	CO5 (BT3)	Module 5, Section Q5.5
Q1 (f)	List and explain various layouts in Java GUI.	5 Marks	CO6 (BT2)	Module 6, Section Q6.3
Q2 (a)	Discuss features of Java language in brief.	5 Marks	CO1 (BT2)	Module 1, Section Q1.1
Q2 (b)	Write a program to print Fibonacci series using recursion.	5 Marks	CO2 (BT3)	Module 2, Section Q2.6
Q2 (c)	Write a program to check whether a string is palindrome or not.	5 Marks	CO3 (BT3)	Module 3, Section Q3.4
Q2 (d)	Discuss importance of Inheritance. List all types of Inheritance and explain any two.	5 Marks	CO4 (BT2)	Module 4, Section Q4.1
Q2 (e)	Explain multiple try and catch with example.	5 Marks	CO5 (BT2)	Module 5, Section Q5.2
Q2 (f)	Differentiate between AWT and Swing.	5 Marks	CO6 (BT4)	Module 6, Section Q6.1
Q3 (a)	What is constructor overloading? Write a program to demonstrate constructor overloading.	10 Marks	CO2 (BT5)	Module 2, Section Q2.4
Q3 (b)	Volume of sphere using Multilevel Inheritance (Radius accepts r → Circle area → Sphere volume).	10 Marks	CO4 (BT4)	Module 4, Section Q4.3
Q3 (c)	University sports grace marks application using interface and base class.	10 Marks	CO4 (BT6)	Module 4, Section Q4.7

Paper 6B: 2413CEC3T1 — Computer Engineering (QP Code: 39724301 | Max Marks: 100)

Q.NO.	QUESTION DESCRIPTION	MARKS	CO & BLOOM	GUIDE SOLUTION REFERENCE
Q1 (a)	Write short note on JDK.	4 Marks	CO1 (BT3)	Module 1, Section Q1.5
Q1 (b)	Explain the different access specifier in Java.	4 Marks	CO2 (BT2)	Module 2, Section Q2.2
Q1 (c)	State the difference between String and StringBuffer class of Java.	4 Marks	CO3 (BT3)	Module 3, Section Q3.3
Q1 (d)	Write a program to demonstrate use of interface in Java.	4 Marks	CO4 (BT3)	Module 4, Section Q4.6 & Q4.7
Q1 (e)	Discuss different layouts of Swing package.	4 Marks	CO6 (BT3)	Module 6, Section Q6.3
Q2 (a)	Compare and contrast C++ and Java.	5 Marks	CO1 (BT2)	Module 1, Section Q1.3
Q2 (b)	Class Rectangle: calculate area and perimeter using constructor.	5 Marks	CO2 (BT6)	Module 2, Section Q2.4
Q2 (c)	Elaborate concept of String class with its any four methods.	5 Marks	CO3 (BT3)	Module 3, Section Q3.3
Q2 (d)	Describe use of 'final' keyword using Java with an example.	5 Marks	CO4 (BT3)	Module 4, Section Q4.4
Q2 (e)	What is concept of exception handling in Java? Discuss try and catch keywords.	5 Marks	CO5 (BT2)	Module 5, Section Q5.1 & Q5.2
Q2 (f)	Write a Java program to print Fibonacci series using recursion.	5 Marks	CO2 (BT3)	Module 2, Section Q2.6
Q3 (a)	What are four pillars of object-oriented programming?	5 Marks	CO1 (BT4)	Module 1, Section Q1.1
Q3 (b)	What are different types of variables used in Java?	5 Marks	CO2 (BT3)	Module 2, Section Q2.3
Q3 (c)	Write a Java program to accept m*n matrix and display it in natural form.	5 Marks	CO3 (BT3)	Module 3, Section Q3.2
Q3 (d)	What is single inheritance with an example?	5 Marks	CO4 (BT3)	Module 4, Section Q4.1 & Q4.2
Q3 (e)	What is thread in Java? How is multithreading implemented in a program?	5 Marks	CO5 (BT3)	Module 5, Section Q5.4
Q3 (f)	Differentiate AWT and Swing class of Java.	5 Marks	CO6 (BT3)	Module 6, Section Q6.1
Q4 (a)	Abstract class Solid: implement Cylinder and Sphere by inheriting from Solid to find volume.	10 Marks	CO4 (BT3)	Module 4, Section Q4.6
Q4 (b)	Illustrate the concept of multithreading in Java with thread lifecycle.	10 Marks	CO5 (BT2)	Module 5, Section Q5.4
Q4 (c)	Elaborate Argument Passing Mechanism? Write a program for different cases with output.	10 Marks	CO2 (BT4)	Module 2, Section Q2.5

LAST-MINUTE REVISION MASTER CHEAT SHEET

1. The 12 Golden Java Keywords & Rules of Thumb

KEYWORD	CORE SEMANTIC MEANING	PRIMARY APPLICATION / RESTRICTIONS
`class`	User-defined blueprint / type.	Can have fields, methods, constructors. Only one `public class` per `.java` file.
`this`	Reference to current object instance.	Differentiates instance fields from method parameters; invokes overloaded constructors via `this()`.
`super`	Reference to immediate superclass.	Calls superclass constructor `super()`; accesses hidden superclass variables and overridden methods.
`static`	Class-level member (shared across all instances).	Stored in Metaspace. Cannot use `this` or `super`; cannot access non-static fields directly without an instance.
`final`	Immutability / constant modifier.	Variable → constant value. Method → cannot be overridden. Class → cannot be inherited.
`abstract`	Incomplete specification.	Abstract classes cannot be instantiated; abstract methods have no body and must be overridden by concrete subclasses.
`interface`	100% abstract contract (pure specification).	Methods are `public abstract` by default; fields are `public static final` constants. Supports multiple realization.
`try-catch`	Exception protection and handling.	Subclass catch blocks must precede superclass catch blocks to prevent unreachable code compilation errors.
`finally`	Guaranteed cleanup code block.	Executes always, even after `return`; skipped only if `System.exit(0)` is invoked. Ideal for closing streams/scanners.
`throw` vs `throws`	Exception triggering vs declaration.	`throw` explicitly instantiates and raises an error; `throws` declares possible checked exceptions in method signature.
`synchronized`	Mutual exclusion thread lock.	Ensures only one thread accesses a critical method or block at a time, preventing race conditions and data corruption.
`package` / `import`	Namespace grouping and access.	`package` must be statement #1 in the source file; `import` brings external classes into the current file namespace.

2. Common Exam Pitfalls & How to Avoid Them

AVOID LOSING MARKS TO THESE 6 FREQUENT MISTAKES

- **Pitfall 1 (Constructor Return Type):** Never write a return type (not even `void`) for constructors. Writing `void MyClass()` turns it into a normal method that is never invoked on object creation!
- **Pitfall 2 (Multiple Catch Ordering):** Placing `catch(Exception e)` above `catch(ArithmeticException e)` causes a fatal compilation error (unreachable catch block). Always order catch blocks from the most specific subclass to the most general superclass.
- **Pitfall 3 (String Equality vs Reference):** Never use `==` to compare String contents. `==` compares memory references (heap addresses); always use `s1.equals(s2)` for character-by-character value comparison.
- **Pitfall 4 (Thread start() vs run()):** Calling `t.run()` executes the code sequentially on the caller thread like a standard method call! You must call `t.start()` to spawn a separate OS thread and trigger the JVM to invoke `run()` asynchronously.
- **Pitfall 5 (super() Position):** `super()` or `this()` constructor calls MUST be the very first statement inside a constructor body. Any statement placed above them triggers an immediate compilation error.
- **Pitfall 6 (Array Length vs String Length):** In Java, array length is a final property accessed without parentheses (`arr.length`), whereas String length is a method call that requires parentheses (`str.length()`).